Tutorial 1: Condition Code Flags (Basic)

Sunday, 25 August 2024 22:33

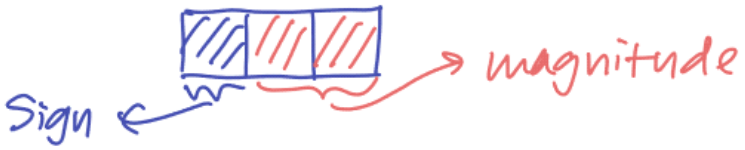
(T)EE2028
Tutorial 1...

Condition Code Flags

- Condition code flags in a status register.
  * flags are updated after every arithmatic/logic operation.

	=1 if:
N – negative	result -ve
Z – zero	result zero
C – Carry	carry-out occurs (unsigned int)
V – overflow	overflow occurs (signed int)

- Signed vs. Unsigned:
  How to rep. negative numbers? 2's complement.
  eg. 3-bit Binary System:



000		0	100		-4
001	⇒	1	101	⇒	-3
010		2	110		-2
011		3	111		-1

From -3 to 0b'100:
1. ignore -ve: 011
2. flip: 100
3. +1: 101

From 0b'111 to -1
1. -1: 110
2. flip: 001
3. Read magnitude

Condition Code Flags

* - Carry: Indicates wrong unsigned result.

E.g. (4 bits)

Operation:

```
  1 1 1 1
+ 0 0 1 0
---------
1 0 0 0 1
```

↳ carry bit

Signed Interpretation: $(-1)+2=1$ ✓

Unsigned Interpretation: $15+2=1$ ✗

* - Overflow: Indicates wrong signed result:

E.g. (4 bits)

Operation:

```
  0 0 1 0
+ 0 1 1 1
---------
  1 0 0 1
```

Signed Interpretation: $2+7=-7$ ✗

Unsigned Interpretation: $2+7=9$ ✓

- Check all 4 flags: 0xF8 + 0x07

Written in binary:

```
  1 1 1 1 1 0 0 0
+ 0 0 0 0 0 1 1 1
---------------
1 1 1 1 1 1 1 1
```

Signed Interpretation: $(-8)+7=-1$ ✓

V = 0

Unsigned Interpretation: $248+7=255$ ✓

C = 0

negative: look at N = 1

zero: since N=1, Z must be 0.

Tutorial 2: Condition Code Flags (Advanced)

Wednesday, 28 August 2024 7:15 pm

Tutorial_2
_Conditio...

More on Cond. Flags.

1. Update of Flags
   - Language dependant
   - In our ARMv7E-M,
     - Arithmetic Inst.: Suffix S ! eg. ADDS, MULS
     - Compare Inst.: No need suffix S. e.g. CMP, CMN.

2. Signed or Unsigned ?

   Not known to the computer ! They are just 1s & 0s, until being used.

   To update N, flag, the program treat the data as signed, or else meaningless.

   But, if you, the programmer knows that you're dealing with Unsigned int.s, you don't care about N.

3. Substraction in hardware (Not examable)

   To perform A − B in hardware is to perform:

   A + 2's(B) ≡ A + 1's(B) + 1.

   1's(B): All bits flipped.

E.g.

Pen n' Paper

```
  1 0 1 0
- 0 1 1 1
---------
  0 0 1 1
```

Hardware

```
  1 0 1 0
+ 1 0 0 0   +1
---------
1 0 0 1 1
```

Carry can happen when performing SUB!

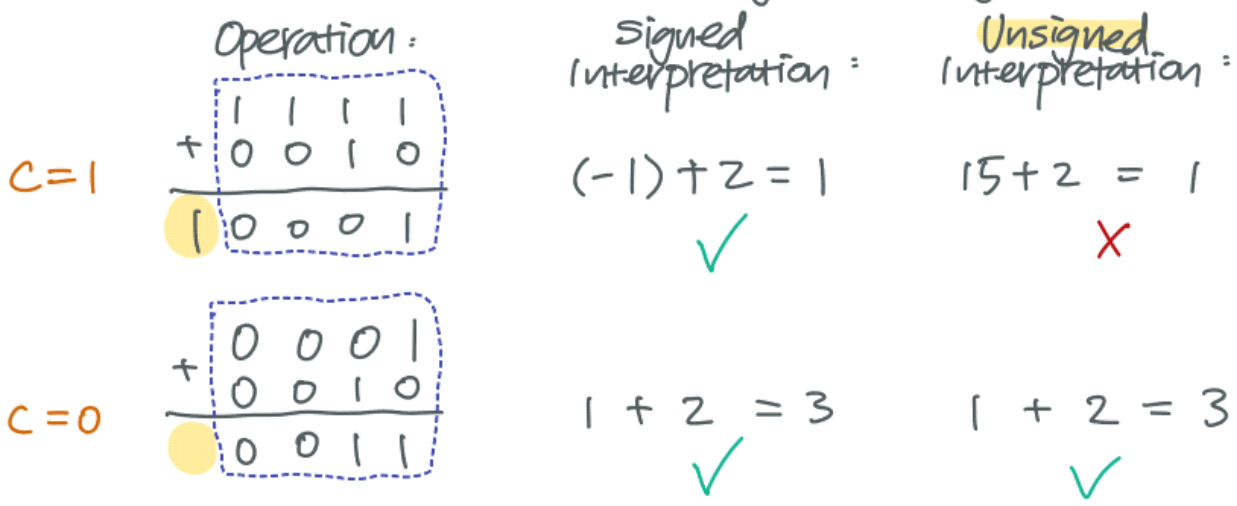
Read again yourself!

3. When exactly will C be 1?

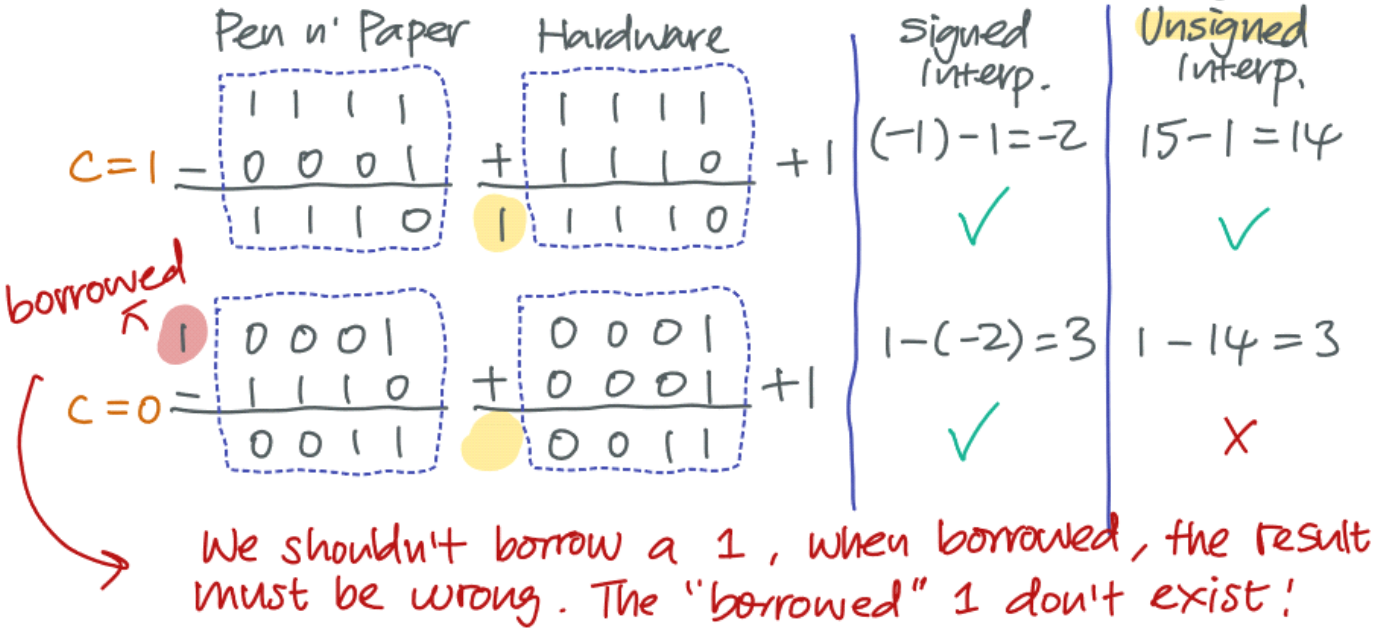
Carry occurs when the extra bit "1" appears.

→ Use C as an indicator of wrong Unsigned result:

I. Addition: C=1 → Unsigned wrong.



II. Substraction: C=0 → Unsigned wrong



We shouldn't borrow a 1, when borrowed, the result must be wrong. The "borrowed" 1 don't exist!

Borrow is another effective way to check if carry happens during SUB, ARM implements carry flag as: not (borrow) for SUB.

3. When exactly will V be 1?

Overflow (V) occurs when the calculated result go against arithmetic rules.

→ Use V as an indicator of wrong signed result:

I. Addition: V=1 → signed wrong.

Occur when: { (−) + (−) = (+) ; (+) + (+) = (−) }

	Operation:	Signed Interpretation:	Unsigned Interpretation:
V=1	0 0 1 0 + 0 1 1 1 = 1 0 0 1	2 + 7 = −2 (+) + (+) = (−) ✗	2 + 7 = 9 ✓
V=1 (C=1)	1 0 1 0 + 1 0 0 1 = (1) 0 0 1 1	(−6) + (−7) = 3 (−) + (−) = (+) ✗	10 + 9 = 3 ✗

II. Substraction: V = 1 → signed wrong

Occur when: { (−) − (+) = (+) ; (+) − (−) = (−) }

	Pen n' Paper	Hardware	Signed Interp.	Unsigned Interp.
V=1 (C=1)	1 0 1 0 − 0 1 1 1 = 0 0 1 1	1 0 1 0 + 1 0 0 0 + 1 = (1) 0 0 1 1	(−6) − 7 = 3 (−) − (+) = (+) ✗	10 − 7 = 3 ✓
borrowed ↖ V=1	(1) 0 0 1 0 − 1 0 0 1 = 1 0 0 1	0 0 1 0 + 0 1 1 0 + 1 = 1 0 0 1	2 − (−7) = −7 (+) − (−) = (−) ✗	2 − 9 = 9 Carry! ✗

Tutorial 2: Instruction Key Points

Wednesday, 28 August 2024 9:48 pm

- Addressing Mode
- Pseudo Instruction
- Compare instructions
- **Branch instructions**

Tutorial_2_ Instructio...

More on Instructions

1. Be careful with #imm size.
   Recall: (Almost) all instruction are restricted to a word long. including the # imm.
   - imm 8 → MOV, CMP, ...
   - imm 12 → ADD, SUB
   - imm 16 → MOVW
   - Do not support imm → MUL, DIV, MLA & variants

2. MEM txf instructions & Addressing Mode
   4 Variations:
   Assume, Rn contains Addr 0xA0, #offset = n
   - LDR/STR Rd, [Rn, #offset] → offset
     After LDR/STR: Rn [0xA0], Rd [Data from 0xA0+n]
   - LDR/STR Rd, [Rn, #offset]! → Pre-ind.
     1. Offset stage: before LDR/STR:
        Rn [0xA0+n], Rd [Undefined]
     2. Perf. LDR/STR
        Rn [0xA0+n], Rd [Data from 0xA0+n]
   - LDR/STR Rd, [Rn], #offset → Post-ind.
     1. Perf. LDR/STR
        Rn [0xA0], Rd [Data from 0xA0]
     2. Offset stage: before LDR/STR:
        Rn [0xA0+n], Rd [Data from 0xA0]
   - LDR Rd, LABEL. → PC-relative
     Assembler knows where label is, and the offset ↑ between current PC [Addr] and label.
     R15 (PC) [Addr of current Inst. + #offset ↑] → Calculated by assembler.
     Rd [Data from label]
     LABEL: .word Addr ← line in code.

More on Instructions

2. (Cont.)

- A special case : Psendo – Instruction (LDR only)

Instead of { LDR Rd, LABEL
⋮
LABEL: .word 0xA123B456

Psendo - Instruction: LDR Rd, = 0xA123B456 (32 bits!)

Wait, anything wrong here?

→ When executing this line, it's splitted into 2 lines.

3. Compare Instructions

CMP / CMN vs. SUBS / ADDS

↓ Update flag

↓ Update flags & Store the result.

4. Branch Instructions:

② BL{cond} LABEL , ③ BLX{cond} Rm

④ BX{cond} Rm

```
Addr: main:
i       ADDS  R2, R2
i+4     BL{Cond} Func
i+8     SUB   R6, R7
i+12  Func:
i+16    ADDS  R2, R3
i+20    BX{Cond} LR
```

After execute this:
if Cond == True,
PC [i+12]
LR [i+8]

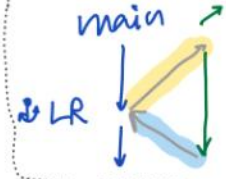
the Addr of the line were to be executed, if not branching

```
Addr: main:
i       ADDS  R2, R2
i+4     BLX{Cond} R8
i+8     SUB   R6, R7
i+12  Func:
i+16    ADDS  R2, R3
i+20    BX{Cond} LR
```

Before execute:
R8 [i+12]
After execute this:
if Cond == True,
PC [i+12]
LR [i+8]

Before execute:
LR [i+8]
After execute this, if Cond == True:
PC [i+8]

BL/BLX BX



LR marks an anchor point to go back to, used when a fuction is called in main.

① B{cond} LABEL.

```
Addr: main:
i       ADDS  R2, R2
i+4     B{Cond} Func
i+8     SUB   R6, R7
i+12  Func:
i+16    ADDS  R2, R3
```

After execute this:
if Cond == True,
PC [i+12]
LR [unchanged]

Most used: if-else, switch-case, loop

Observe the difference: Last e.g., we can't go back to i+8 with BX.

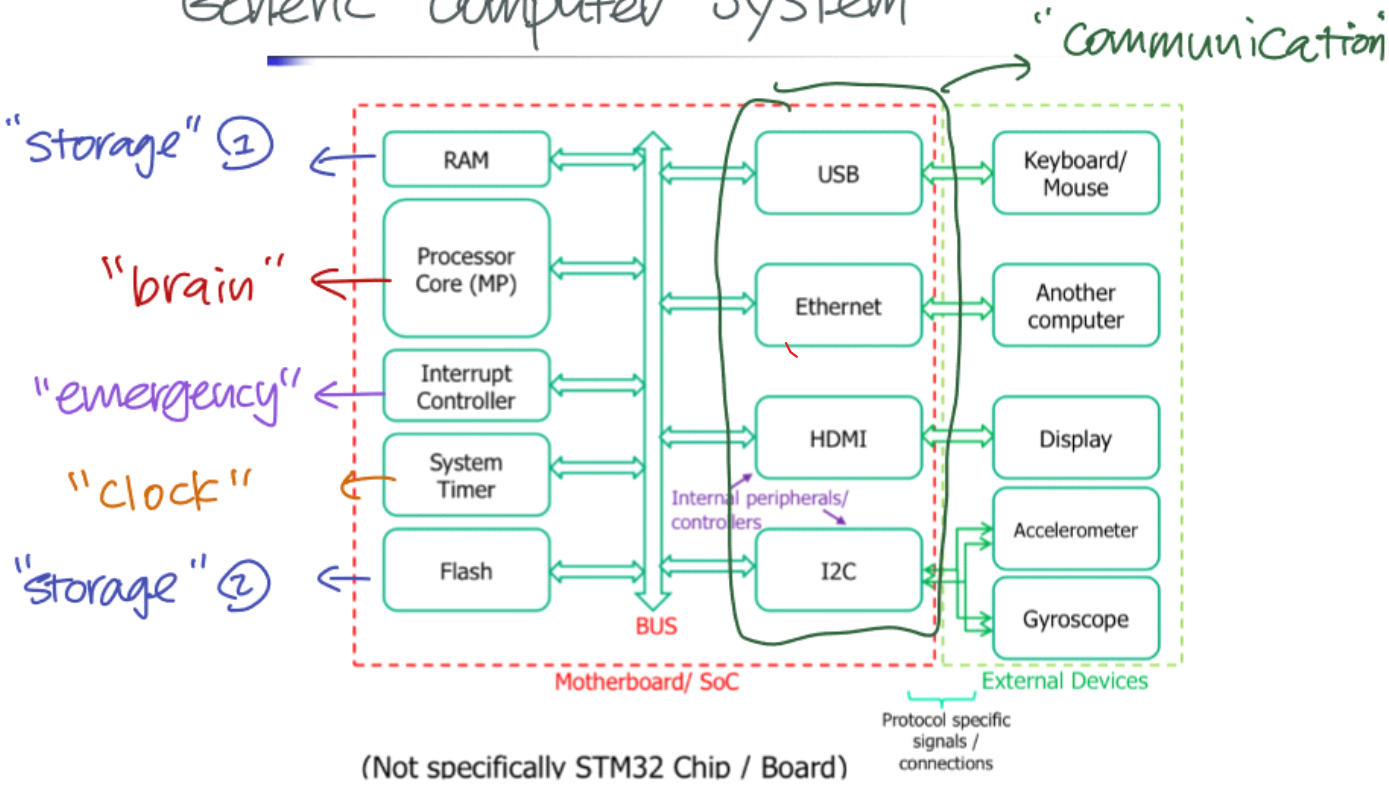
Tutorial 4: System Architecture/ Protocols Overview/ GPIO Summary

Sunday, 6 October 2024 2:23 pm

Tutorial_4
_System ...

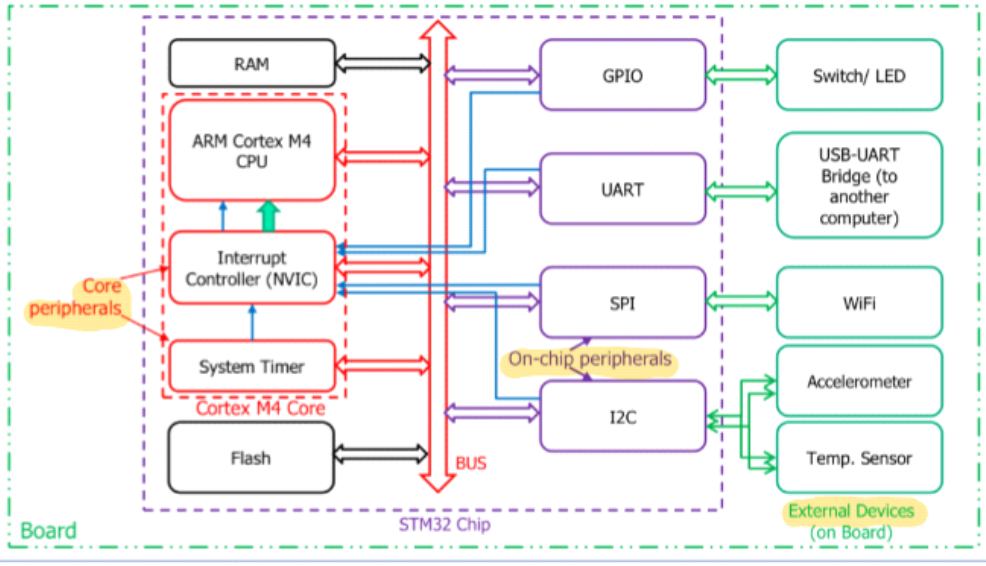
Summary: Interfacing & GPIO

- Generic Computer System



- RAM : volatile, fast, truly random access.
  Flash : Non-volatile, slow, randomly/sequentially.

- STM32 Chip & Board (Partial)

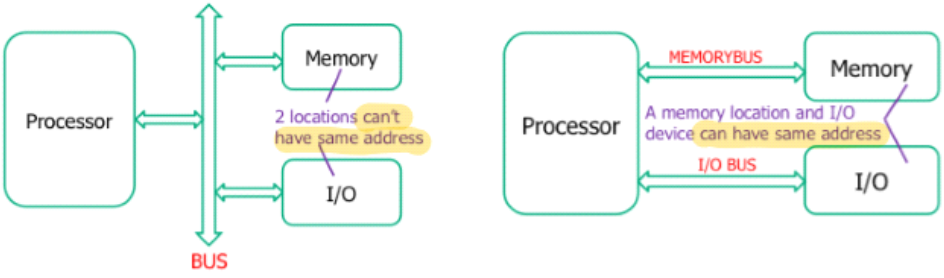


- Libraries / API / drivers
  - → CMSIS : core peripherals
  - → HAL : on-chip peripherals (protocol)
  - → BSP : external peripherals (sensors)

— Communicate with hardwares —

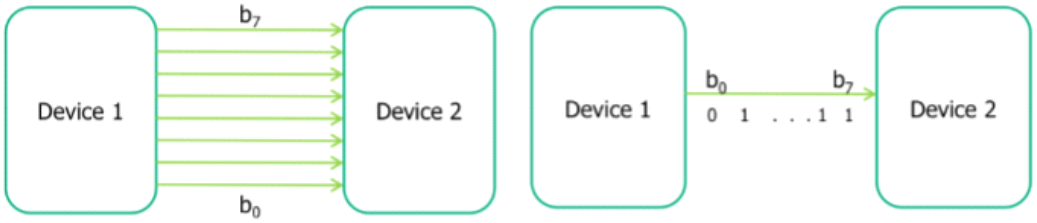
- Peripheral Registers
  - → How / what to read / write? → Datasheet.
  - → Type of P.R:
    - Status : read by processor
    - Control : written by processor
    - Data : read or write for data txf.

- MMIO vs. PMIO



- → MMIO: Shared addr. space, unique address.
  LDR/STR equivalent for all.
- → PMIO : Memory & Perip. in seperated addr space.
  - LDR/STR equivalent for memory.
  - IN/OUT for perip.

- Parallel vs. Serial Protocals:



Prallel: { "old" version
- fast/efficient with short ranges.
- cross-talk, synchronization issue, ↓ bus-skew.
- poor scalability, etc.

Serial: {
- Compact
- Solves most of the issue listed.
- However!! Still deal with data parallelly!

- Synchronous v.s. Asynchronous Protocals:

Sync: {

SPI
I2C
- Common time reference
- fast
- cross-talk, skew.

Async: {

UART
USB
- rcvr needs to recover timing info.
- slower,
- complicated hardware.

- Bus-based v.s. Point-to-Point

  - Bus-based: (I^2C, SPI, USB)
    - multiple devices share a bus.
    - each have unique addr.
  - Point-to-point (UART)
    - Dedicated link between 2 devices.
    - No addressing required.

- Master-Slave v.s. Peer-to-Peer

  - Master-slave: (I^2C, SPI, USB, AHB)
    - Master initiate comm.
    - -- decides which slave can respond.
    - generate clk (in sync protocal)
  - Peer-to-peer: Any device can initiate comm.
    - Ethernet (configuration dependent)

- Simplex, Half Duplex & Full Duplex:

  - Simplex: single, unidirectional link, one-way.
    - GPIO O/P to LEDs, I/P from switches.
  - Half Duplex: one bi-directional link, take turns.
    - I^2C, USB 2.0
  - Full Duplex: At least 2 links, tx/rx at the same time.
    - SPI, UART, USB 3.0.

- In-band v.s. Out-of-band Protocals:

In-band: addr. & data thru same bus (selection of device)

Out-of-band: Seperate addr. bus, a decoder enables devices.

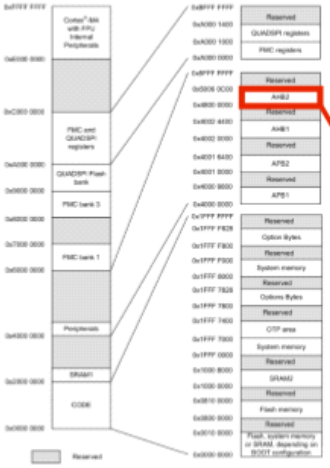
- Summary:

Protocal	Parallel v.s. Serial	Sync v.s Async.	Bus-based v.s. Point 2 point	Master-Slave vs. Peer 2 Peer	Simpl/ Half/ full Duplex	In-band/ out-of band
UART	S	Async	P2P	P2P (typically)	Full	In.
I2C	S	Sync	Bus	M-S	Half	In.
SPI	S	Sync	Bus	M-S	Full	Out
USB	S	Async	Bus	M-S	2.0 Half 3.0 Full	In.

GPIO.

- General purpose Input / Output
- GPIO is an interface.
  - → Each pin can be configured individually.
  - → physical pin can detect / set a voltage.
  - → Do not have a defined protocal
- Can be used to implement protocals (I2C, UART, ...)

- GPIO Registers:



Memory map for STM32L475xx devices

Bus	Boundary address	Size (bytes)	Peripheral
AHB2	0x4800 1C00 - 0x4800 1FFF	1 KB	GPIOH
	0x4800 1800 - 0x4800 1BFF	1 KB	GPIOG
	0x4800 1400 - 0x4800 17FF	1 KB	GPIOF
	0x4800 1000 - 0x4800 13FF	1 KB	GPIOE
	0x4800 0C00 - 0x4800 0FFF	1 KB	GPIOD
	0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC
	0x4800 0400 - 0x4800 07FF	1 KB	GPIOB
	0x4800 0000 - 0x4800 03FF	1 KB	GPIOA

STM32L475xx devices peripheral register boundary addresses

Register Type	Register Name (where x = A to H)		Read-Write
Control	Mode register	GPIOx_MODER	RW
	Output type register	GPIOx_OTYPER	RW
	Output speed register	GPIOx_OSPEEDR	RW
	Pull-up / Pull-down register	GPIOx_PUPDR	RW
	Configuration lock register	GPIOx_LCKR	RW
	Alternate function low register	GPIOx_AFRL	RW
	Alternate function high register	GPIOx_AFRH	RW
	Analog switch control register	GPIOx_ASCR	RW
Data	Input data register	GPIOx_IDR	RO
	Output data register	GPIOx_ODR	RW
	Bit reset register	GPIOx_BRR	WO
	Bit set/reset register	GPIOx_BSRR	WO

Ref. to RM0351_STM32L4x5 MCUs Ref Manual, Pg75-77

- General Idea:
  - → Each port (A, B, ..., H) contain a set of Rs.
  - → Each R in the set contains 32 bits
  - → Each pins requires a subset of bits from a R / or multiple Rs to be configured.

    e.g. MODER: 2 bits per pin, in total 1 R,

    AFRL/H: 4 bits per pin, in total 2 Rs.

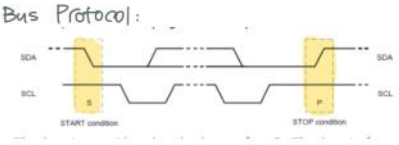
Tutorial 5: I2C Protocol

2024年10月13日 15:36

Tutorial_5_I2C Protocol

I²C

- Signals
  - → Two bi-directional lines.
    - Serial Data line SDA
    - Serial Clk line SCL.
  - → When free, both line High due to pull-up.
  - → Output devices connected → open-drain
- I²C Bus Protocol:



→ Transaction: Start with "S", end with "P".

→ All devices will keep listening to the bus, is it free?

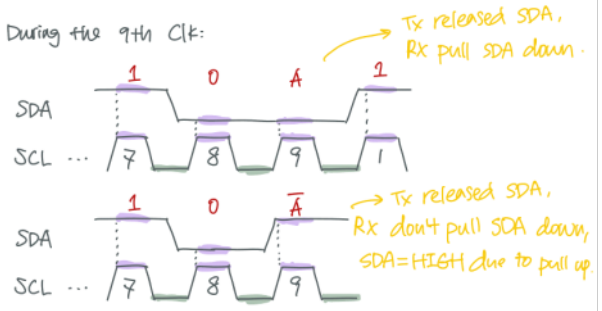
→ { S, P: SDA change when SCL is High.
  { Data: SDA change when SCL is Low.

→ Slave addressing: 7 bits, followed by R/W (1/0).

→ Data tx: { Every byte → 8 bits on SDA
  { No limitation on bytes per transfer.

→ ACK: Receiver acknowledge every byte (8 bits).
Every 9th clk, { Tx release SDA to High
  { Rx pull SDA to Low.
  { SDA remain Low during high clk period indicate ACK (A).
  { SDA remain High → NACK ($\overline{A}$).

During the 9th Clk:



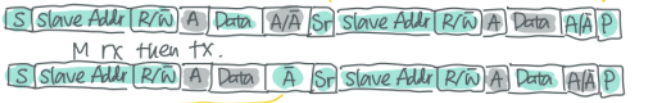
- Master Tx:

- Master Rx:

→ Always begin with M sending S Addr.
→ A send, B Ack. B send, A Ack.
→ Ā from M: "I don't want anymore data".

- Change of direction

M tx then rx:



M rx then tx.

↳ Master-rx sends Ā just before Sr.

- NACK:

Usually Ā from the slave has no standard meaning.

With Ā on SDA, master can:
- generate "P", abort the txf.
- generate "Sr", start new txf.
- resend previous byte.

- Clock Stretching:

When the slave is not ready to rx/tx,

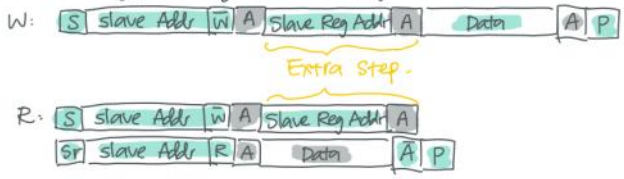
⇒ hold SCL Low and force Master to wait.

⇒ The clk signal is generated by the Master, but since the slave is stretching the clk to LOW, the clk won't be generated.

⇒ The Master will notice this and wait.

⇒ This is the only case when the slave controls SCL.

- Writing / Reading I²C Slave Register.



Welcome to EE2028 Tutorial!

Wednesday, 14 August 2024 4:34 pm

Some housekeeping

- About me: **Qingqing**, a full-time instructor in the ECE Department.
- Feel free to stop me anytime to ask questions.
- **Cooling-off period**: I DO NOT answer any questions 24 hours prior to midterm, and final. If you have questions, ask EARLY.
- Contact me through my favourite turn-based strategy game: ✉ ni.qq@nus.edu.sg

Access the Note

- Log in with your **NUS ID**.
- Copy the link below and paste in your browser to open the notebook in OneNote App.
- https://nusu-my.sharepoint.com/:o:/g/personal/ni_qq_nus_edu_sg/EkL5HkVTF_5KmdWbG-Lzg8cBuol5sezaRBHYdJ24y77elA

Tutorial 01: Microprocessor Concepts

Sunday, 25 August 2024 5:24 pm

(T)EE2028 Tutorial 1...

National University of Singapore
Dept of Electrical & Computer Engineering (ECE)

[T]EE2028 Microcontroller Programming & Interfacing

Tutorial 1: Microprocessor Concepts

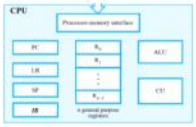
Note: "Add", "Load" and "Store" used in this tutorial are **generic** assembly language instructions.

1. Given a binary pattern in some memory location, is it possible to tell whether this pattern represents a *machine instruction* or a *number*?

2. List the *steps needed* to execute the machine instruction

Add R4, R2, R3

in terms of simple control commands and the information (i.e. instruction or data) transfers between the processor components and memory discussed in Lecture 1, as shown on the right. Assume that the address of the memory location containing this instruction is initially in register PC.



3. A program to add a number stored at location A to another number stored at location B on a RISC computer is given by:

```
Load   R2, A
Load   R3, B
Add    R4, R2, R3
```

Explain why the program cannot simply be written as:

Add R4, A, B

4. A program comprising several instructions is stored in the main memory at the locations shown below. In order to repeat the execution of the sequence of instructions (found in locations 0x00000100 to 0x0000010C) when a certain condition is satisfied, a conditional branch instruction (at 0x00000110) is used in the program.

What is the effect of the conditional branch instruction on the *content of the Program Counter (PC) register* in the processor when the condition is satisfied (i.e. the whole sequence of 4 instructions is to be repeated)?

Main Memory	
0x00000100	Load R2, A
0x00000104	Load R3, B
0x00000108	Add R4, R2, R3
0x0000010C	Store R4, C
0x00000110	Conditional branch
0x00000114	:

5. Regarding the *program counter (PC)*,

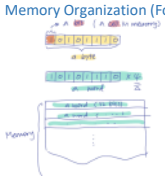
- The flow of an ARM assembly program can be altered by changing the content of the PC.
  [True / False]
- By default, the PC is incremented by 4 because the next instruction word is 4 bits away in Cortex-M4.
  [True / False]

END

Q1. No.

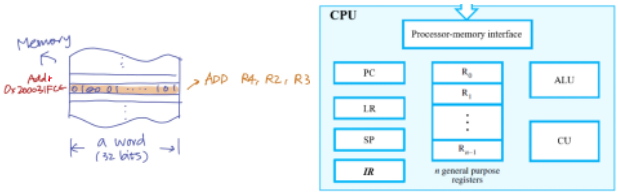
- Information is stored in the form of 0s and 1s. (*When using higher-level languages, the compiler translates source code into machine code that a computer's processor can execute.*)+9
- Memory: bit/ **byte**/ word (byte is the smallest addressable unit)
- A word may store:
  - (part of) an instruction
  - Data (float, string, integer, long, ...)
- Any binary pattern can be interpreted as an instruction or as a number

Memory Organization (For 32-bit ARM)



Q2.

- Instruction **ADD R4, R2, R3** is stored in memory -> can be located with an address
- Assume this address is 0x2000 31FC, the steps for executing the instruction are:



1. Program Counter (**PC**) should contain the address before executing this instruction:
   0x2000 31FC
2. Control Unit (**CU**) sends address in **PC** to **Processor-memory interface**, issue *Read* command.
3. **CU** loads the instruction into the Instruction Register (**IR**), now **IR** contains the instruction in the forms of machine code (0s and 1s).
4. **CU** decodes the machine code, to determine the operation
5. **CU** *sends* the content of **R2** and **R3** and *issues* **ADD** command to Arithmetic & Logic Unit (**ALU**)
6. **CU** gets the output from **ALU** and sends it to **R4**.
7. **PC ++**, pointing to the next instruction in the memory address:
   0x2000 3200

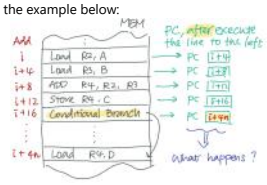
Q3.

- In a RISC computer, only **Load** and **Store** instructions are used to access memory operands.
- All the **operands** for arithmetic/logic instructions must be in the **registers**, or **one of them** may be a constant given explicitly in the instruction word, as #immediate.
- In this case, A and B are both memory locations.

Q4.

The Conditional Branch instruction could lead to branching or looping.

- When executing instructions *in sequence*, by default, the *value* (address pointing to the next instruction) stored in the **PC** increases by 4.
- **Branching**: At the conditional branch line, it checks the condition given and *places* the address of the next instruction to be executed into **PC**.
  - The value contained in PC might *increase* by 4n instead of 4!
    See the example below:



- **Looping**: Similarly, when looping is instructed, the address pointing to the first instruction of the looped section will be *placed* into the **PC**.
  - The value contained in PC might *decrease* by 4n!

Read on your own

More on conditional branching/ looping: **Condition Code Flags**

Q5.

- The flow of an ARM assembly program can be altered by changing the content of the PC. (T)
- By default, the PC is incremented by 4 because the next instruction word is 4 ~~bits~~ bytes away in Cortex-M4. (F)

Tutorial 02: ARMv7E-M Assembly Language

Sunday, 25 August 2024 10:39 pm

CG2028 Tutorial 2...

Solns

Solution password: 24102028

National University of Singapore
Dept of Electrical & Computer Engineering (ECE)

Don't worry Tut 2 for CG and EE are identical. I'm just being lazy.

CG2028 Computer Organization

Tutorial 2: ARMv7E-M Assembly Language

1. Which of the following ARM instructions will cause the assembler to issue a syntax error message? Explain why?

(a)	ADD	R2, R2, R2
(b)	SUB	R0, R1, [R2, #4]
(c)	MOV	R0, R1, R2
(d)	LDR	R0, R1
(e)	ADDS	R0, R1, R2, LSL #3
(f)	BLXAL	LOL
(g)	MVNS	R1, R1 → Bitwise NOT & update flag
(h)	EOR	R1, R1, R1 → Bitwise exclusive OR
(i)	RRX	R1, R1 → Rotate Right with extend.
(j)	TEQS	R1, R2
(k)	PUSH	{R0-R16}
(l)	POPS	{R0, R1, R2}

Read Datasheet.

2. Assume the following register and memory contents in an ARM embedded system. What is the effect of executing the following instructions with the given initial values?

Main Memory	
0x00000100	**LDR R8, [R0]**
0x00000104	**LDR R9, [R0, #4]**
0x00000108	**ADD R10, R8, R9**
:	:
0x00001000	1
0x00001004	2
0x00001008	3
0x0000100C	4
0x00001010	5
0x00001014	6
:	:

Registers	
PC	
:	
R0	0x1000
R1	0x2000
R2	0x1010
R3	0x300
:	:
R8	0x400
R9	0x500
R10	0x600
:	:

3. Assume the following register and memory contents in an ARM embedded system. What is the effect of executing the following instructions with the given initial values?

Main Memory	
0x00000100	**STR R6, [R1, #-4]!**
0x00000104	**STR R7, [R1, #-4]!**
0x00000108	**LDR R8, [R1], #4**
0x0000010C	**LDR R9, [R1], #4**
0x00000110	**SUB R10, R8, R9**
:	:
0x00001000	1
0x00001004	2
0x00001008	3
0x0000100C	4
0x00001010	5
0x00001014	6

Registers	
R0	0x1000
R1	0x2000
R2	0x1010
R3	0x300
R4	0x200
R5	10
R6	20
R7	30
R8	0x400
R9	0x500
R10	0x600
:	:

4. Write an ARM assembly program that finds the number of negative integers in a list of *n* 32-bit integers and stores the count in location NEGNUM. The value *n* is stored in memory location N, and the first integer in the list is stored in location NUMBERS.
Hint: use the IF-THEN (IT) block.

5. Write an ARM assembly program to reverse the order of the bits in register R2. For example, if the

Q1. Instruction Syntax

Erroneous instructions:

(b). [Rd] is a memory operand, arithmetic instructions should not access memory locations.

(c). **MOV** is a one-to-one operation, inclusion of a second source operand (R2) is invalid.

(d). **LDR** (and **STR**) are for data transfer between register and memory locations. To transfer data between registers, use **MOV**.

(f). **BLXAL** ***LOL***-> **BLX**{*Cond.*} ***LABEL*** (This is to break down the given instruction, not a valid syntax)
BLX (Branch Indirect with Link register) must be used with ***Rm***.
Rm should contain the branch target address.

(j). **TEQ** (Test if EQual). Test instructions update condition flags without the need to specify the **S** suffix.

(k). In ARMv7E-M, there are only 16 directly-accessible registers, R0 to R15.

(l). The **S** suffix is invalid for **PUSH**/ **POP**.

[Read more on branching instructions in additional note]

Correct instructions that appear to be wrong:

(e). **ADDS R0**, R1, R2, **LSL** #3
Destination. 2nd Op. 1st Op.

For R2, **LSL** #3:
- R2: base register
- **LSL**: Logical Shift Left
- #3: No. of positions to be shifted

This instruction performs: R0 = R1 + (R2 * 8)

Q2. Addressing Mode

Main Memory	
0x00000100	**LDR R8, [R0]**
0x00000104	**LDR R9, [R0, #4]**
0x00000108	**ADD R10, R8, R9**
:	:
0x00001000	1
0x00001004	2
0x00001008	3
0x0000100C	4
0x00001010	5
0x00001014	6
:	:

Registers	
PC	
:	
R0	0x1000
R1	0x2000
R2	0x1010
R3	0x300
:	:
R8	~~0x400~~ 1
R9	~~0x500~~ 2
R10	~~0x600~~ 3
:	:

0x1000 + 4 → 0x1004

- R0 contains the address of a memory location
- **LDR** R9, [R0, #4]
  - #4 is an one-time offset
  - Do **not** change the value stored in the root register (R0 in this case)
  - Equivalent to **LDR** R9, =0x0000 1004 (Pseudo-Instruction)

[Read more in additional note]
- Pseudo-instruction
- Addressing mode

Q3. Addressing Mode

Main Memory	
0x00000100	**STR R6, [R1, #-4]!**
0x00000104	**STR R7, [R1, #-4]!**
0x00000108	**LDR R8, [R1], #4**
0x0000010C	**LDR R9, [R1], #4**
0x00000110	**SUB R10, R8, R9**
:	:
0x00001000	1
0x00001004	2
0x00001008	3
0x0000100C	4
0x00001010	5
0x00001014	6
:	:

Registers	
R0	0x1000
R1	~~0x2000~~ ~~0x1FFC~~ 0x1FF8 ....
R2	0x1010
R3	0x300
R4	0x200
R5	10
R6	20
R7	30
R8	~~0x400~~ 30
R9	~~0x500~~ 20
R10	~~0x600~~ 10
:	:

0x00001FFF8	30
0x00001FFC	20
0x00002000	

- R1 contains the address of a memory location
- **STR** R6, [R1, #-4]!
  - Pre-indexing -> Offset then STR
  - The value stored in the root register is offset by #-4 (pay attention to the !)
  - Equivalent to
    - **ADD** R1, #-4
    - **STR** R6, [R1]
    - Pseudo-Instruction does not work with STR
- **LDR** R8, [R1], #4
  - Post-indexing -> LDR then Offset
  - The value stored in the root register is offset by #4
  - Equivalent to
    - **ADD** R1, #4
    - **LDR** R8, [R1]

Q4. If-Then(-Else) & Loop

Given in Memory:

Address	Data
...	...
N	n
...	...
NUMBERS	1st integer in the list

and stores the count in location NEGNUM. The value *n* is stored in memory location N, and the first integer in the list is stored in location NUMBERS.
Hint: use the IF-THEN (IT) block.

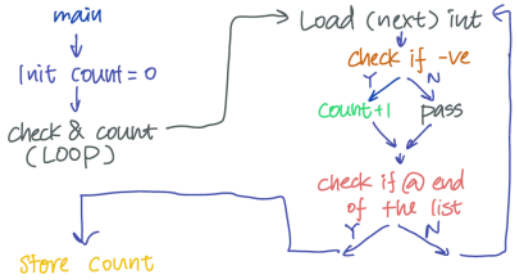
5. Write an ARM assembly program to reverse the order of the bits in register R2. For example, if the starting pattern in R2 is 1110…0100, the final result in R2 should be 0010…0111.
Hint: use shift and rotate operations.

6. Write an ARM assembly program that generates the first *n* numbers of the Fibonacci series. In this series, the first two numbers are 0 and 1, and each subsequent number is generated by adding the preceding two numbers. For example, for *n*=8, the series is 0, 1, 1, 2, 3, 5, 8, 13. Your program should store the numbers in successive memory word locations starting at MEMLOC. Assume that the value *n* is stored in location N.
Hint: assumes that the last number in the series of *n* numbers can be represented in a 32-bit word, and that *n* >2.

END

...	...
N	n
...	...
NUMBERS	1st integer in the list

Draft the logic flow:



Analyse:
- We are to implement 1 loop and 2 IF-THEN structures.
- 2 ways to implement the check & count loop
  - As a loop (simple, with **B**{*cond*})
  - As a subroutine (slightly challenging, with **BL**/**BLX**{*cond*} and **BX**{*cond*})

Sample code implemented a simple loop:

```
init  LDR   R1, =NUMBERS      @ Load address of data list
      LDR   R2, N             @ Load size of list
      MOV   R0, #0            @ Clear negative number counter

LOOP: LDR   R3, [R1], #4      @ Load next data item
      CMP   R3, #0            @ Set condition code flags
      IT    MI                @ Start of IT block
      ADDMI R0, #1            @ Increment counter if data
                              item is negative
      SUBS  R2, #1            @ Decrement loop counter
      BNE   LOOP              @ Branch back if not done
      LDR   R4, =NEGNUM
      STR   R0, [R4]          @ Store result
```

[Read more about Condition Flag in additional note]

Q5. Branch

Hint: If you need to look for an instruction, go to ARMv7-M Architecture Ref Manual.pdf, which can be found on Canvas.

A7.7.67 LSL (immediate)

Logical Shift Left (immediate) shifts a register value left by an immediate number of bits, shifting in zeros, and writes the result to the destination register. It can optionally update the condition flags based on the result.

Encoding T1 All versions of the Thumb instruction set.

LSLS <Rd>,<Rm>,#<imm5>	Outside IT block.
LSL<c> <Rd>,<Rm>,#<imm5>	Inside IT block.

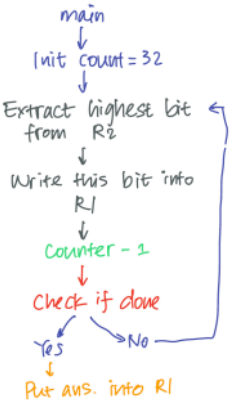
A7.7.116 RRX

Rotate Right with Extend provides the value of the contents of a register shifted right by one place, with the carry flag shifted into bit<31>.

RRX can optionally update the condition flags based on the result. In that case, bit<0> is shifted into the carry flag.

Encoding T1 ARMv7-M

RRX{S}<c> <Rd>,<Rm>




```
Init. MOV   R0, #32           @ Load R0 with count value 32
LOOP:  LSLS  R2, R2, #1       @ Shift contents of R2 left one
                              bit position, setting the high-
                              order bit into the C flag
       RRX   R1, R1           @ Rotate R1 right one bit
                              position, including the C flag
       SUBS  R0, #1           @ Check if finished
       BNE   LOOP
       MOV   R2, R1           @ Load reversed pattern back
                              into R2
```

Tutorial 03: C Programming

Tuesday, 3 September 2024 15:34

(T)EE2028 Tutorial 3 ...

<-
Solution password: 24102028

National University of Singapore
Dept of Electrical & Computer Engineering (ECE)

[T]EE2028 Microcontroller Programming & Interfacing

Tutorial 3: C Programming

1. Which of the following is/are not allowed in C and may cause syntax error(s)? Explain why.
   (a) int a, main(void);
   (b) bool led_status = true;
   (c) char c = NULL;
   (d) #define TEE = 2028;
   (e) int16_t ee 2028;
   (f) printf("%s\n", z? "good" : "ok" : "bad");
   (g) z? a = 10 : (y? a = 1 : (a =0)); //variables z and y are bool, and a is int
   (h) printf("Percentage = %d%%\n", weightage);
   (i) #define circum(r,pi) (2*pi*r) → function-like Macro.
   (j) return sum; //function returns sum to main()

2. Write a C program that counts the number of negative integers in a list of *N* 32-bit integers. The list is given in an array *list[ ]*. Your program should store the count in a variable *negcnt*, in addition to extracting all the negative integers and storing them in an array *negnum[ ]*.
   Hint: use the for loop and *if* constructs.

3. Write a C program to reverse the order of the 8 bits in a character and add 30 to it for cross-checking. For example, if the input is ASCII character 'T' (0b01010100), the reversed order is 0b00101010, i.e., ASCII character ' * '. A character 'H' should be obtained when 30 is finally added to this reversed character.
   Hint: use shift and bitwise logic operations.

4. In the Fibonacci series, the first two integers are 0 and 1, and each subsequent integer is generated by adding the preceding two integers. For example, for *n*=8, the series is 0, 1, 1, 2, 3, 5, 8, 13. Write a C program that calls a Fibonacci function that generates the series and passes the result back to main() for printing.
   Hint: Use pass by reference, e.g., declare the function as **void fibonacci(int n, int *fibo)**, where *n* is the number of integers (assume *n* >2) and *fibo* is a pointer to the array that stores the generated series.

5. Two one-dimensional integer arrays, A and B, both have *N* elements. Write a simple C program that uses pointers to find the sum of A and B and stores the result in A, i.e.

   A[0]=A[0]+B[0], A[1]=A[1]+B[1], …, A[N-1]=A[N-1]+B[N-1].

END

Q1. Instruction Syntax

Erroneous lines:

(a). **int a, main(void)**
- Put the two of them together -> it's ok!
- Missing the semicolon ";" -> not ok!
- Not a usual approach but it is indeed valid to declare the main together with other variables
- The common practice is to perform declaration and definition together.

(c). **char c=NULL;**
NULL is usually defined as **(void *0)**, which is a null pointer instead of a character. Type mismatch.

(d). **#define TEE=2028;**
correct syntax: #define TEE 2028

(e). **int16_t ee 2028;**
correct syntax: int16_t ee=2028;

(f). **printf("%s\n", z? "good" : "ok" : "bad");**
- For the ternary operator **?** :" takes in one condition and two expressions.

Correct lines that could use some explanation:

(g). **z? a = 10 : (y? a = 1 : (a = 0)); //variables z and y are bool, and a is int.**
Two nested ternary operations, refer to the flowchart to understand how it works ->

(h). **printf("Percentage = %d%%\n", weightage);**
%d -> format specifier
%% -> use %: to escape format specifier
2nd %: to print a "%"

Q2. Loop, If Statement, Print array/list with for-loop

Code highlights
- For loop:

```
for (int i=0; Stop Condition; i++){
    do these until hit the stop condition
    …
}
```

- If Statement

```
if (Condition) {
    do these if the Condition checks
    …
}
```

```c
#include <stdio.h>

int main() {
    const int N=5;
    int list[]= {1,2,-3,-4,-5};
    int negcnt = 0;   //Counter for number of neg numbers
    int negnum[N];    //Array to store neg numbers

    for (int i=0; i<N; i++){
        if(list[i]<0){
            negnum[negcnt] = list[i];
            negcnt++;
        }
    }

    printf("There are %d neg numbers\n", negcnt);
    printf("The neg num are:\n");
    for (int i=0; i<negcnt; i++){
        printf("%d \n", negnum[i]);
    }

    return 0;
}
```

```
/tmp/ATgaWTmraS.o
There are 3 neg numbers
The neg num are:
-3
-4
-5
```

Q3. Ternary operator, Loop, ASCII, Bit Shift, Logical operations

We are asked to write a program that:
1. Converts an ASCII character to its ASCII code,
2. Reverse the ASCII code order
3. Add a 30
4. Convert the resulting code back to an ASCII character

I've provided a different (possibly harder to understand) version of the solution here, as it involves some useful skills for interfacing with hardware through C.

```c
#include <stdio.h>

unsigned char reverseBits(unsigned char x) {
    unsigned char reversed = 0;
    for (int i = 0; i < 8; i++) {
        // Shift reversed to the left and add the least significant bit of x
        reversed = (reversed << 1) | (x & 1);
        // Shift x to the right for the next iteration
        x >>= 1;
    }
    return reversed;
}

void printBinary(unsigned char x) {
    for (int i = 7; i >= 0; i--) {
        putchar((x & (1 << i)) ? '1' : '0');
    }
}

int main() {
    char inputChar = 'T';
    unsigned char reversedChar = reverseBits(inputChar);
    char resultChar = reversedChar + 30;

    printf("Original char: %c\n", inputChar);
    printf("Binary of input char: ");
    printBinary(inputChar);
    putchar('\n');

    printf("After reversing bits and adding 30: %c\n", resultChar);
    printf("Binary of result char: ");
    printBinary(resultChar);
    putchar('\n');

    return 0;
}
```

```
/tmp/AIgeWTmraS.o
Original char: T
Binary of input char: 01010100
After reversing bits and adding 30: H
Binary of result char: 01001000
```

Make sure to also check out another interesting approach provided in the answer sheet.

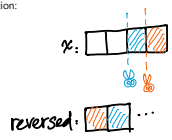
Understand the core logic (Two lines highlighted)

(Similar idea with Tut 2, Q5)
Let's assume ***x*** = 0101, and ***reversed*** = 0000

Ln 7: `reversed = (reversed << 1) | (x & 1);` (1 ≡ 0001)
- `(reversed << 1)`: shift reserved left, pad with a '0'.
  - 1st pass: ***reversed*** = 0000 << 1 = 0000,
  - 2nd pass: ***reversed*** = 0001 << 1 = 0010.
- `(x & 1)`: bitwise AND, retrieve the rightmost bit of x
  - 1st pass: ***x*** = 0101, ***x*** & 0001 = 0001
  - 2nd pass: ***x*** = 0010, ***x*** & 0001 = 0000
- `reversed = …|…` : bitwise OR, sets the LSB of reversed to the LSB of x retrieved.
  - 1st pass: ***reversed*** = 0000 | 0001 = 0001
  - 2nd pass: ***reversed*** = 0010 | 0000 = 0010

Ln 9: `x >> = 1;`
- Shift x to the right by 1 bit for the next round.
  - 1st pass: ***x*** = 0101 >> 1 = 0010
  - 2nd pass: ***x*** = 0010 >> 1 = 0001

In human language: "Cut" the rightmost bit of **x**, and append it to the rightmost of **reversed**.
The shifting left of **reversed** is to make room
The shifting right of **x** is to facilitate the "cut"
With diagram illustration:



National University of Singapore
Dept of Electrical & Computer Engineering (ECE)

[T]EE2028 Microcontroller Programming & Interfacing

Tutorial 3: C Programming

1. Which of the following is/are not allowed in C and may cause syntax error(s)? Explain why.
   (a) int a, main(void)
   (b) bool led_status = true;
   (c) char c = NULL;
   (d) #define TEE = 2028;
   (e) int16_t ee 2028;
   (f) printf("%s\n", z? "good" : "ok" : "bad");
   (g) z? a = 10 : {y? a = 1 : {a =0} }; //variables z and y are bool, and a is int
   (h) printf("Percentage = %d%%\n", weightage);
   (i) #define circum(r,pi) (2*pi*r) → function-like Macro.
   (j) return sum; //function returns sum to main()

2. Write a C program that counts the number of negative integers in a list of *N* 32-bit integers. The list is given in an array *list[ ]*. Your program should store the count in a variable *negcnt*, in addition to extracting all the negative integers and storing them in an array *negnum[ ]*.
   Hint: use the for loop and *if* constructs.

3. Write a C program to reverse the order of the 8 bits in a character and add 30 to it for cross-checking. For example, if the input is ASCII character 'T' (0b01010100), the reversed order is 0b00101010, i.e., ASCII character '*'. A character 'H' should be obtained when 30 is finally added to this reversed character.
   Hint: use shift and bitwise logic operations.

4. In the Fibonacci series, the first two integers are 0 and 1, and each subsequent integer is generated by adding the preceding two integers. For example, for *n*=8, the series is 0, 1, 1, 2, 3, 5, 8, 13. Write a C program that calls a Fibonacci function that generates the series and passes the result back to main() for printing.
   Hint: Use pass by reference, e.g., declare the function as **void fibonacci(int n, int *fibo)**, where *n* is the number of integers (assume *n* >2) and *fibo* is a pointer to the array that stores the generated series.

5. Two one-dimensional integer arrays, A and B, both have *N* elements. Write a simple C program that uses pointers to find the sum of A and B and stores the result in A, i.e.

   A[0]=A[0]+B[0], A[1]=A[1]+B[1], …., A[N-1]=A[N-1]+B[N-1].

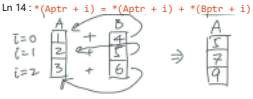
END

Q5. Declare and use pointers

```c
#define N 3

void fibonacci(int, int*);

int main() {
    int A[N] = {1, 2, 3};
    int B[N] = {4, 5, 6};

    int *Aptr, *Bptr;
    Aptr = A;
    Bptr = B;

    for(int i=0; i<N; i++){
        *(Aptr+i) = *(Aptr+i)+*(Bptr+i);
    }
    for(int i=0; i<N; i++){
        printf("%d\n", *(Aptr+i));
    }
    return 0;
```

Output:
```
/tmp/2k0ngvYZyC.o
5
7
9
```

- Ln 9 : `int *Aptr, *Bptr;`
  - This line declares two **empty** pointers.
    - *What's the difference between these two pointers and the array pointer in Q4?*
    - You can make `Aptr` and `Bptr` point to any variable, but the array name can only be used to point to the array itself (thus the "constant").
- Ln 10 : `Aptr = A;`
  - Assigned the pointer `Aptr` to the address of the 1st element of the array `A`.
    - `A`, as the name of an array, can be treated as a pointer.
    - Equate `Aptr` to `A`, making `Aptr` a pointer to the 1st element of the array `A`.
- Ln 14 : `*(Aptr + i) = *(Aptr + i) + *(Bptr + i)`



I heard that your brain still exists: Are there any redundant lines here in the code?

```c
#define N 3

void fibonacci(int, int*);

int main() {
    int A[N] = {1, 2, 3};
    int B[N] = {4, 5, 6};

    for(int i=0; i<N; i++){
        *(A+i) = *(A+i)+*(B+i);
    }
    for(int i=0; i<N; i++){
        printf("%d\n", *(A+i));
    }
    return 0;
}
```

Output:
```
/tmp/2k0ngvYZyC.o
5
7
9
```

Anyone still remember that the array name can be treated as a pointer as well? Then why bother declaring that two empty pointers?

Q4. Pointers, dereference (*)

```c
#define N 10

void fibonacci(int, int*);

int main() {
    int fibo_store[N];
    fibonacci(N, fibo_store);
    for(int i=0; i<N; i++){
        printf("%d\n",fibo_store[i]);
    }
    return 0;
}

void fibonacci(int n, int *fibo){
    *(fibo) = 0;
    *(fibo+1) = 1;
    for(int i=2; i<n; i++){
        *(fibo+i)=*(fibo+i-1)+*(fibo+i-2);
    }
    return;
}
```

Declare ①, Call ③, Define ②

Output:
```
/tmp/2k0ngvYZyC.o
0 1 1 2 3 5 8 13 21 34
```

We shall focus on Ln 6, Ln 14 - Ln 21:

- Ln 6: `int fibo_store[N];`
  - This line defines an array and allocates (blocks) enough memory space for N integers.
  - The array name `fibo_store`, is ***functionally equivalent*** to a constant pointer, that points to the starting address of this block of memory.

- Ln 14: `void fibonacci (int n, int * fibo)`
  - `int * fibo ->` emphasizes that the second argument is a pointer
  - This line is *equivalent* to `void fibonacci (int n, int fibo[])` (why?)
    - Array in C naturally decays into a pointer, when passing the array around, we are passing the pointer to the first element, this is constant regardless of how is this array being passed.
- Ln 15-16: `*(fibo) = 0; *(fibo + 1) = 1;`
  - `*(fibo) = 0;` is equivalent to `fibo[0] = 0;`
  - `*(fibo + 1) = 0;` is equivalent to `fibo[1] = 0;`
  - ***Why 1?*** The + 1 here represents "increment by **one position** in the array". When performing pointer arithmetic like fibo + 1, the compiler automatically accounts for the size of the data type being pointed to. (That is to say, it moves to 4 bytes later)

Let me fry your brain a bit deeper: Will the following two versions of code work?

```c
#include <stdio.h>
#define N 10

void fibonacci(int, int*);

int main() {
    int fibo_store[N];
    fibonacci(N, fibo_store);
    for(int i = 0; i < N; i++) {
        printf("%d ", fibo_store[i]);
    }
    return 0;
}

void fibonacci(int n, int *fibo) {
    *(fibo) = 0;
    fibo[1] = 1;
    for(int i = 2; i < n; i++) {
        *(fibo + i) = *(fibo + i - 1) + *(fibo + i - 2);
    }
    return;
}
```

```c
#include <stdio.h>
#define N 10

void fibonacci(int, int*);

int main() {
    int fibo_store[N];
    fibonacci(N, fibo_store);
    for(int i = 0; i < N; i++) {
        printf("%d ", fibo_store[i]);
    }
    return 0;
}

void fibonacci(int n, int fibo[]) {
    *(fibo) = 0;
    *(fibo+1) = 1;
    for(int i = 2; i < n; i++) {
        *(fibo + i) = *(fibo + i - 1) + *(fibo + i - 2);
    }
    return;
}
```

Let me cook your brain beyond well-done so that it is now congratulations: What is the fundamental difference between passing array and array pointer into function in C? (int *fibo vs. int fibo[])
https://stackoverflow.com/questions/5573310/difference-between-passing-array-and-array-pointer-into-function-in-c

The perfect steak

WOLF
@COOLWOLF

This isn't even well done this is congratulations

Tutorial 04: Interfacing Concepts and GPIO

Sunday, 6 October 2024 2:03 pm

[T]EE2028 Tutorial 4 Interfacing Concepts and GPIO

Answer

[T]EE2028 Tutorial 4 ...

National University of Singapore

Dept of Electrical & Computer Engineering (ECE)

[T]EE2028 Microcontroller Programming & Interfacing

Tutorial 4: Interfacing Concepts and General Purpose Input-Output (GPIO)

Note: Note all questions will be discussed in the tutorial. Treat the rest as practice questions. Written solutions will be provided.

1. Name an interface characterized by the terms master-slave, bus-based, synchronous, serial, half-duplex.
2. Port B pin 14 of STM32L475VG has an LED connected to it externally. Write an ARMv7M assembly language code to configure the MODE of this pin appropriately such that the other pins in the port are not affected by the code.
3. In addition to the connection in the question above, a push button BTN1 is connected to STM32L475VG port A pin 10. BTN1 is active low, and LED is active high. Assuming that the pins are already configured to be in the correct mode, write a C program to turn on the LED whenever BTN1 is pressed.
4. Use HAL driver function(s) to configure the pins for LED and BTN1 appropriately for the question above.
5. An altimeter* provides the altitude above sea level in the range [-50 m, 150 m] as a signed 12-bit value, in the form of 2 separate bytes with the format $A_{11}A_{10}A_9A_8$xxxx (MSB) and $A_7A_6A_5A_4A_3A_2A_1A_0$ (LSB). If the sensor gives MSB = 0xA8 and LSB = 0xCD, what is the altitude in metres? Note: x's in the format denotes reserved bits / don't cares – the values from these bits should be ignored.
6. For the scenario in Q.5 above, implement a function `void print_altitude()` that prints the altitude in metres. You can assume that a function `void altitude_read(char* altitude_val_raw)` is provided which reads the altimeter and writes the two bytes corresponding to the altitude value into the location specified as `altitude_val_raw`, MSB first.

* There are a few different types of altimeters. Explore the principles and use cases!

National University of Singapore

Dept of Electrical & Computer Engineering (ECE)

[T]EE2028 Microcontroller Programming & Interfacing

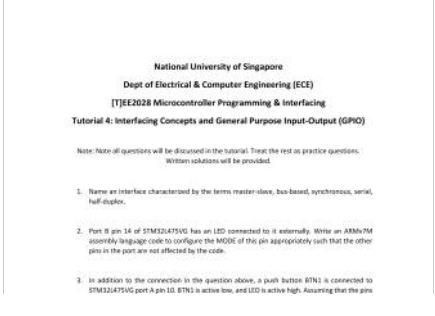
Tutorial 4: Interfacing Concepts and General Purpose Input-Output (GPIO)

Note: Note all questions will be discussed in the tutorial. Treat the rest as practice questions. Written solutions will be provided.

1. Name an interface characterized by the terms master-slave, bus-based, synchronous, serial, half-duplex.
2. Port B pin 14 of STM32L475VG has an LED connected to it externally. Write an ARMv7M assembly language code to configure the MODE of this pin appropriately such that the other pins in the port are not affected by the code.
3. In addition to the connection in the question above, a push button BTN1 is connected to STM32L475VG port A pin 10. BTN1 is active low, and LED is active high. Assuming that the pins are already configured to be in the correct mode, write a C program to turn on the LED whenever BTN1 is pressed.
4. Use HAL driver function(s) to configure the pins for LED and BTN1 appropriately for the question above.
5. An altimeter* provides the altitude above sea level in the range [-50 m, 150 m] as a signed 12-bit value, in the form of 2 separate bytes with the format $A_{11}A_{10}A_9A_8$xxxx (MSB) and $A_7A_6A_5A_4A_3A_2A_1A_0$ (LSB). If the sensor gives MSB = 0xA8 and LSB = 0xCD, what is the altitude in metres? Note: x's in the format denotes reserved bits / don't cares – the values from these bits should be ignored.
6. For the scenario in Q.5 above, implement a function `void print_altitude()` that prints the altitude in metres. You can assume that a function `void altitude_read(char* altitude_val_raw)` is provided which reads the altimeter and writes the two bytes corresponding to the altitude value into the location specified as `altitude_val_raw`, MSB first.

* There are a few different types of altimeters. Explore the principles and use cases!

© Gu Jing and Rajesh Panicker, ECE, NUS



National University of Singapore

Dept of Electrical & Computer Engineering (ECE)

[T]EE2028 Microcontroller Programming & Interfacing

Tutorial 4: Interfacing Concepts and General Purpose Input-Output (GPIO)

Note: Note all questions will be discussed in the tutorial. Treat the rest as practice questions. Written solutions will be provided.

1. Name an interface characterized by the terms master-slave, bus-based, synchronous, serial, half-duplex.
2. Port B pin 14 of STM32L475VG has an LED connected to it externally. Write an ARMv7M assembly language code to configure the MODE of this pin appropriately such that the other pins in the port are not affected by the code.
3. In addition to the connection in the question above, a push button BTN1 is connected to STM32L475VG port A pin 10. BTN1 is active low, and LED is active high. Assuming that the pins

Q1. Classifying Hardware Protocols

Answer: **I2C**.

Protocol	Parallel vs Serial	Sync vs Async	Bus-based vs Point-2-Point	Master-Slave vs Peer-2-Peer	Simplex / Half / Full Duplex	In-band vs Out-of-band
UART	S	Async	P-2-P	P-2-P (Simplex)	Full	In.
I2C	S	Sync	Bus	M-S	Half	In.
SPI	S	Sync	Bus	M-S	Full	Out
USB	S	Async	Bus	M-S	2.0 Half 3.0 Full	In.

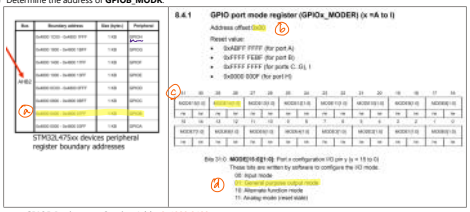
* *Taken from Additional Notes, read more here.*

Q2. GPIO (Control) Registers

Key Steps to configure the MODE of PB14.

(1) Identify that to configure MODE -> Write to **MODER**

(2) Determine the address of **GPIOB_MODR**.



a. **GPIOB** Registers -> Starting Addr.: 0x4800 0400

b. **GPIOx_MODER** Addr. Offset -> 0x00

c. **GPIOB_MODER** Addr. = Base Addr. + Addr. Offset = 0x4800 0400 + 0x00 = 0x4800 0400

(3) Determine the bits in control of **Pin 14**.

a. **GPIOx_MODER** PIN 14 -> MODE14[1:0] (GPIOx_MODER[29:28])

(4) Determine the **values to be written**.

d. LED -> Output -> General purpose output mode: write the value "0b01"

```
.equ GPIOB_MODER, 0x48000400   -> Addr. of R.
LDR R1, =GPIOB_MODER           } load 32 bits from R
LDR R0, [R1]
BIC R0, R0, 0x30000000         -> clear
ORR R0, R0, 0x10000000         -> write
STR R0, [R1]                   -> store
```

Q3. GPIO (Data) Registers

Key Steps to Read Input Peripheral (BTN1, PA10) / Set Output Peripheral (LED, PB14) Logic Level

(1) Plan pseudocode

```
if (BTN1 NOT pressed)
  {LED Off};
else // pressed.
  {LED On;}
```

(2) Check BTN1 (PA10) Logic Level

Register: GPIO Data Register -> Input Data Register -> **GPIOA_IDR**

GPIOA Registers -> Starting Addr.: 0x4800 0000

GPIOA_IDR Addr. = Base Addr. + Addr. Offset = 0x4800 0000 + 0x10 = 0x4800 0010

PIN(s): **GPIOA_IDR**[10]

(3) Set LED (PB14) Logic Level

Registers: GPIO Data Register -> Output Data Register -> ~~**GPIOB_ODR**~~ **GPIOB_BSRR** **GPIOB_BRR**

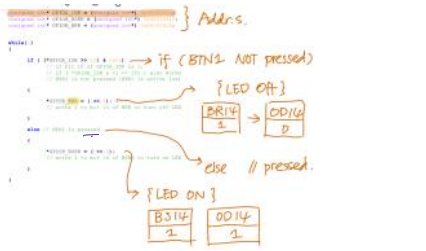
* *Set/Reset **GPIOx_ODR** through **GPIOx_BSRR/BRR** for better efficiency, directly writing to **GPIOx_ODR** requires reading 32 bits and writing the updated 32 bits.*

GPIOB_BSRR Addr. = 0x4800 0418

GPIOB_BRR Addr. = 0x4800 0428

PIN(s): **GPIOB_BSRR**[30] (Reset with '0b1') & **GPIOB_BSRR**[14] (Set with '0b1')

GPIOB_BRR[14] (Reset with '0b1')



Type Casting

In the first three lines, we created 3 pointers and have them pointing to the corresponding registers.

```
unsigned int* GPIOA_IDR = (unsigned int*) 0x48000010;
```

The highlighted type casting is applied to the value in GPIOA_IDR, this is to inform the compiler to treat the value as an unsigned integer.

Q4. GPIO Initialization/Configuration (Through HAL Driver Functions), Understanding `GPIO_InitTypeDef`

My "monkey see, monkey do approach": Analyze a sample code then adopt its style.

Reference Code: Lab 1 demo code, `blink by HAL` (Focus on highlighted configuration code)

```
21 #include "main.h"
22 static void MX_GPIO_Init(void);
23
24⊖ int main(void)
25 {
26
27   HAL_Init();
28   MX_GPIO_Init();
29
30   while (1)
31   {
32     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
33     HAL_Delay(50);
34   }
35 }
36
37⊖ static void MX_GPIO_Init(void)
38 {
39   GPIO_InitTypeDef GPIO_InitStruct = {0};
40
41   /* GPIO Ports Clock Enable */
42   __HAL_RCC_GPIOB_CLK_ENABLE();
43
44   /*Configure GPIO pin Output Level */
45   HAL_GPIO_WritePin(GPIOB, LED2_Pin, GPIO_PIN_RESET);
46
47   /*Configure GPIO pin : LED2_Pin */
48   GPIO_InitStruct.Pin = LED2_Pin;
49   GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
50   GPIO_InitStruct.Pull = GPIO_NOPULL;
51   GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
52   HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
53 }
```

'Blueprint' — Entity, an empty form

Speed, Pin, Pull, Mode, ...

Filling in the form

Pass the completed form to the processor.

Understanding Structure

- Recall OOP (Object-Oriented Programming) from CS1010E
- In python: Class -> a blueprint, Instance -> an entity
- In C: Structure -> a blueprint, Instance -> an entity
- In the above example, **`GPIO_InitTypeDef`** -> the blueprint, **`GPIO_InitStruct`** -> the entity.

Our Code:

```
GPIO_InitTypeDef GPIO_InitStruct;
// configure port A pin 10 as input
GPIO_InitStruct.Pin = GPIO_PIN_10; // declared in HAL_gpio.c as 0x0400
/* if there is another pin that should have the same settings (say
pin 11, we could do GPIO_PIN_10|GPIO_PIN_11 */
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
// configure port B pin 14 as output
GPIO_InitStruct.Pin = GPIO_PIN_14;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

BTN, PA10 — BTN1 is active low. Set default to high, else always active even if not pushed.

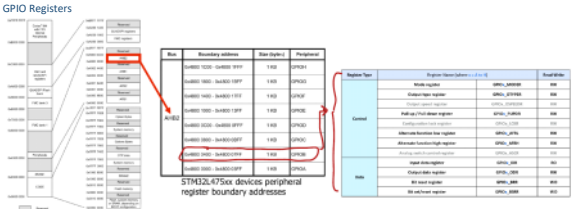
LED, PB14 — LED is an output.

Output Mode: Push-Pull vs. Open-Drain

PP: Sets pin logic level to HIGH -> Output sets to High Voltage (5V/ 3.3V/ 3V, etc.)

Sets pin logic level to LOW -> Output sets to 0V.

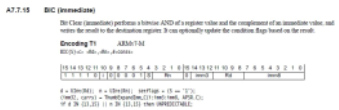
GPIO Registers



- Each port (A, B, ..., H) contains a set of Registers
- Each Register in the set comprises 32-bits
- Each pin requires a subset of bits from a Register or from multiple Registers to be configured
- E.g., **MODER**: 2 bits per pin, in total 1 Register for a port

  AFRL/H: 4 bits per pin, in total 2 Register for a port

BIC (to Clear [29:28])



(1) Complement

0x3000 0000 = 0b0011 0000 ... 0000

↓ Flip All bits

0xCFFF FFFF = 0b1100 1111 ... 1111 (Mask)

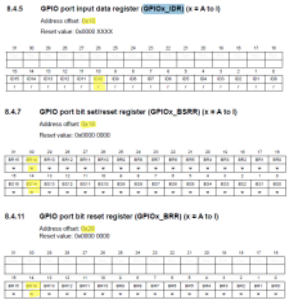
(2) AND

GPIOB_MODER bitwise AND Mask, **clear** bits [29:28], keep other bits unchanged.

ORR (to Write [29:28])

0x1000 0000 = 0b0001 0000 ... 0000

Writes '0b01'.



half-duplex.

2. Port B pin 14 of STM32L475VG has an LED connected to it externally. Write an ARMv7M assembly language code to configure the MODE of this pin appropriately such that the other pins in the port are not affected by the code.

3. In addition to the connection in the question above, a push button BTN1 is connected to STM32L475VG port A pin 10. BTN1 is active low, and LED is active high. Assuming that the pins are already configured to be in the correct mode, write a C program to turn on the LED whenever BTN1 is pressed.

4. Use HAL driver function(s) to configure the pins for LED and BTN1 appropriately for the question above.

5. An altimeter* provides the altitude above sea level in the range [-50 m, 150 m] as a signed 12-bit value, in the form of 2 separate bytes with the format $A_{11}A_{10}A_9A_8xxxx$ (MSB) and $A_7A_6A_5A_4A_3A_2A_1A_0$ (LSB). If the sensor gives MSB = 0xAB and LSB = 0xCD, what is the altitude in metres? Note: x's in the format denotes reserved bits / don't cares – the values from these bits should be ignored.

6. For the scenario in Q.5 above, implement a function `void print_altitude()` that prints the altitude in metres. You can assume that a function `void altitude_read(char* altitude_val_raw)` is provided which reads the altimeter and writes the two bytes corresponding to the altitude value into the location specified as `altitude_val_raw`, MSB first.

* There are a few different types of altimeters. Explore the principles and use cases!


```
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```
→ LED is an output.

Output Mode: Push-Pull vs. Open-Drain

PP: Sets pin logic level to HIGH -> Output sets to High Voltage (5V/ 3.3V/ 3V, etc.)
Sets pin logic level to LOW -> Output sets to 0V.

OD: Sets pin logic level to HIGH -> High impedance (Z), Disconnected.
Sets pin logic level to LOW -> Output sets to 0V.

Q5. Interpreting Sensor Output (Mathematical Calculation)

Key takeaway points:

- Sensitivity = Measurement Range/Bit Width
- Set up an expression in the form of $y = mx + c$
  - Where: m -> Sensitivity,
  - c -> Midway of the measurement range, if the value is signed (this question)
  - c -> Lower bound of the measurement range, if the value is unsigned

Q6. Interpreting Sensor Output (Code Implementation)

Please go through the answer on your own.

```c
#include <stdio.h>

void altitude_read(char*);

void print_altitude() {

    char altitude_val[2];
    altitude_read(altitude_val);

    int altitude_val_concat = ( altitude_val[0]<<8 & 0xFF00 ) |
                              ( altitude_val[1] & 0xFF );
    /* when a signed char/byte is promoted to a signed int, the MSB of
    the byte is duplicated into bits 31:8 (sign extension). These become
    bits 11:0 after <<4. & 0xFFFF0FF0 preserves these bits. */

    printf("Concatenated value (Hex)    : %x\n", altitude_val_concat);
    printf("Concatenated value (Decimal): %d\n", altitude_val_concat);

    float altitude_m = 50 + (float)altitude_val_concat * 200/4096;

    printf("The altitude is %f metres", altitude_m);

}

void altitude_read(char* altitude_val_raw)
{
    /* In reality, the data will be read from the altimeter through an
    interface such as I2C or SPI. We are hard-coding here for testing */
    altitude_val_raw[0] = 0xAB; //MSB
    altitude_val_raw[1] = 0xCD; //LSB
}
```

Interesting question asked:

```c
void altitude_read(char*)
--
char altitude_val[2]
```

Why `char*` ?

From the implementation you can see that we are expecting a pointer to the data, which is in form of **2 bytes**. We thus need to make sure we reserved enough space.

- In C, a **byte buffer** is traditionally `char*` because `char` is the basic character type and is **guaranteed to be size 1 byte** (or 8 bits). Many legacy I²C/SPI APIs were written this way.
  - In that case, why don't we use `uint_8`?
- Corner case: on very odd devices where a "byte" isn't 8 bits, `uint8_t` may not exist, but `char` always does.
- Core reason:
  In C's strict-aliasing model, accessing the same memory through pointers of different, incompatible types can be **undefined behaviour (UB)***. the compiler may assume they don't overlap and optimize in ways that break your program.
  Character types are a special exception: the standard allows any object's bytes to be accessed via `char*`, `signed char*`, or `unsigned char*`.
  Therefore, using a `char*` as the buffer type for raw sensor data is **safe** and cannot cause aliasing UB.
  It's ideal for I²C/SPI reads where we treat values as raw bytes first, and only later reinterpret/assemble them into larger integers.

  *UB means the C standard imposes no requirements on the result—code may seem to work, fail intermittently, or change with compiler flags/optimizations.

Tutorial 05: Inter-Integrated Circuit (I2C) Interface

2024年10月13日 15:37

Answer

[T]EE2028 Tutorial 5 I2C

[T]EE2028 Tutorial 5...

NATIONAL UNIVERSITY OF SINGAPORE
Department of Electrical and Computer Engineering

[T]EE2028 Microcontroller Programming and Interfacing

Tutorial 5

Inter-Integrated Circuit (I²C) Interface

1. Suppose you wish to connect an I²C-based light sensor with the board. Can you connect it to I2C2? If that is not possible, how else can you interface the sensor? Which is/are the STM32 chip pins involved?

Questions 2-6 are based on the `HTS221_T_ReadTemp()` function in hts221.c file.

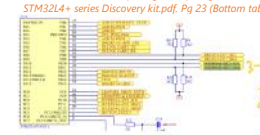
2. Compute the total number of (a) Bytes sent (b) Bytes received (c) START bits sent (d) STOP bits sent (e) Acknowledges sent (f) Not-Acknowledges sent by the STM32 chip while executing this function.
3. What is the significance of '| 0x80' that appears at three different places in the function?
4. Is it possible to make the function more efficient with minimal effort (i.e., without venturing into interrupts etc.), assuming that the temperature sensor is read repeatedly once a second from the main program? A full function implementation is not expected, explaining the idea will do.
5. Assuming the following register contents, compute the temperature reading returned by the function.

T0_degC_x8	0xA5
T1_degC_x8	0x07
T1/T0 msb	0xC4
T0_OUT_L	0xFD
T0_OUT_H	0xFF
T1_OUT_L	0xB3
T1_OUT_H	0x02
TEMP_OUT_L	0x38
TEMP_OUT_H	0x02

6. Draw the voltage levels on SDA2 and SCL2 when `tmp = SENSOR_IO_Read(DeviceAddr, HTS221_T0_T1_DEGC_H2);` within the function is executed, and indicate the START, STOP and ACK conditions. Indicate whether the master or the slave executes the condition at specific moments. Also indicate the points in time at which the various bits of I2C_ISR of STM32 I2C2 peripheral are set. Assume slave (HTS221) register contents as specified in question 5.

Q1. I2C Hardware Readiness

** L4S5VI-e03_schematic.pdf, Pg. 3 (Top diagram)*
STM32L4+ series Discovery kit.pdf, Pg 23 (Bottom table)



Questions 2-6 are based on the HTS221_T_ReadTemp() function in hts221.c file.

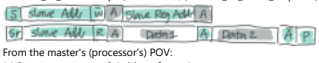
Q2. I2C Transaction Breakdown

Compute the total number of (a) Bytes sent (b) Bytes received (c) START bits sent (d) STOP bits sent (e) Acknowledges sent (f) Not-Acknowledges sent by the STM32 chip while executing this function.

(1) Identify the lines that involve I2C tx/rx:
- i. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_T0_DEGC_X8 | 0x80), buffer, 2);
- ii. tmp = SENSOR_IO_Read(DeviceAddr, HTS221_T0_T1_DEGC_H2);
- iii. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_T0_OUT_L | 0x80), buffer, 4);
- iv. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_TEMP_OUT_L_REG | 0x80), buffer, 2);

(2) Determine the number of data frames (in bytes) to be transmitted:
- i. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_T0_DEGC_X8 | 0x80), buffer, 2);
- ii. tmp = SENSOR_IO_Read(DeviceAddr, HTS221_T0_T1_DEGC_H2); (1 byte)
- iii. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_T0_OUT_L | 0x80), buffer, 4);
- iv. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_TEMP_OUT_L_REG | 0x80), buffer, 2);

(3) Draft the transaction timeline and calculate:
(Here I provided a detailed solution for case i, please work out the rest after class)
- i. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_T0_DEGC_X8 | 0x80), buffer, 2);



From the master's (processor's) POV:
(a) Bytes sent -> 3 (address frames)
(b) Bytes received -> 2 (data frames)
(c) START bits sent -> 2 (S & Sr)
(d) STOP bits sent -> 1
(e) ACK sent -> 1
(f) NACK sent -> 1

Q3. I2C Read/Write Operation Control

What is the significance of '| 0x80' that appears at three different places in the function?

- i. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_T0_DEGC_X8 | 0x80), buffer, 2);
- ii. tmp = SENSOR_IO_Read(DeviceAddr, HTS221_T0_T1_DEGC_H2);
- iii. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_T0_OUT_L | 0x80), buffer, 4);
- iv. SENSOR_IO_ReadMultiple(DeviceAddr, (HTS221_TEMP_OUT_L_REG | 0x80), buffer, 2);

'| 0x80' is used as a part of the slave register address, and if you are unsure what its significance is, the best way to find it out is to read the datasheet. (Google 'HTS221 datasheet')

To navigate the datasheet, use Ctrl + F and search for the keywords I2C/ Slave Address.

On page 15, under section 5.1.1.:

> The I²C embedded in the HTS221 behaves like a slave device and the following protocol must be adhered to. After the start condition (ST) a slave address is sent, once a slave acknowledge (SAK) has been returned, an 8-bit sub-address (SUB) will be transmitted: the 7 LSB represents the actual register address while the MSB enables address auto-increment. If the MSB of the SUB field is '1', the SUB (register address) will be automatically increased to allow multiple data read/write.

| 0x80 is equivalent to | 0b1000 0000, it sets the MSB of the sub-address to '1', and enables auto-increment for reading consecutive registers in a single operation.

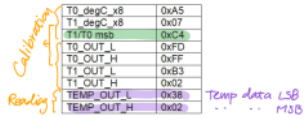
In the function call, we provided the register address of **HTS221_T0_DEGC_X8** but it reads both **HTS221_T0_DEGC_X8** and **HTS221_T1_DEGC_X8**.

Q5. Sensor Data Interpretation from Register Values

Assuming the following register contents, compute the temperature reading returned by the function.

Again, the best way to gather information is to read the datasheet.

To make sense of each register and its significance, read section 7, Register description.



Now, if you reach the end of section 7, you should see section 8 is titled: ' Humidity and temperature data conversion'. This section introduced the use of the register values given.

(1) Making sense of the calibration registers:

Step 1: Temperature conversion from ADC_OUT (LSB) to °C

Data to build the Temperature calibration curve are stored in device registers.

Linear interpolation (example):

Input temperatures LSB (ADC) / Output temperature (°C)

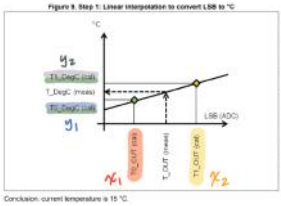
T0_OUT = 300 / (Mid T0_degC || T0_degC)_x8 = 60 °C => 80/8 = 10.0°C

T1_OUT = 500 / (Mid T1_degC || T1_degC)_x8 = 160 °C => 160/8 = 20.0°C

Temperature conversion:

T_OUT = 400 / T_degC_x8 =120 °C =>T_degC=120/8 = 15.0 °C

Figure 9. Step 1: Linear interpolation to convert LSB to °C



Conclusion: current temperature is 15 °C

T0_degC_x8	0xA5
T1_degC_x8	0x07
T1/T0 msb	0xC4
T0_OUT_L	0xFD
T0_OUT_H	0xFF
T1_OUT_L	0xB3
T1_OUT_H	0x02
TEMP_OUT_L	0x38
TEMP_OUT_H	0x02

(2) Find out x1, x2, y1, y2, and x

- Note that y1 and y2 values are not given directly.
- Join **MSB of T0/T1** with **T0/T1_degC_x8** to get the magnified (x8) full value.
- The precise y1 and y2 are given by (full value/8).
- The library file implemented an integer approach, which discards the 3 LSB with '> >3'.

```
T0_degC = T0_degC_x8_u16 >> 3;
T1_degC = T1_degC_x8_u16 >> 3;
```

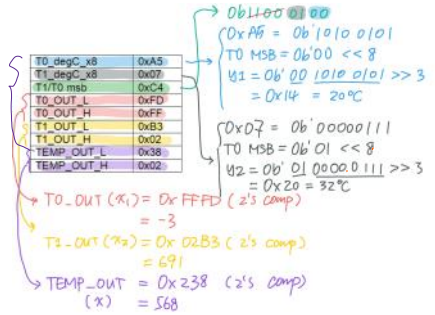
- We may make this implementation more accurate by declaring T0/T1_degC as floats, and replacing'> > 3' with '/8'.

Table 19. Decoding the coefficients in the sensor Flash

Addr	Variable	Format	b7	b6	b5	b4	b3	b2	b1	b0
Output registers										
28	H_OUT	(s16)	H7	H6	H5	H4	H3	H2	H1	H0
29			H15	H14	H13	H12	H11	H10	H9	H8
2A	T_OUT	(s16)	T7	T6	T5	T4	T3	T2	T1	T0
2B			T15	T14	T13	T12	T11	T10	T9	T8
Calibration registers										
30	H0_rH_x2	(u8)	H0.7	H0.6	H0.5	H0.4	H0.3	H0.2	H0.1	H0.1
31	H1_rH_x2	(u8)	H1.7	H1.6	H1.5	H1.4	H1.3	H1.2	H1.1	H1.0
32	T0_degC_x8	(u8)	T0.7	T0.6	T0.5	T0.4	T0.3	T0.2	T0.1	T0.0
33	T1_degC_x8	(u8)	T1.7	T1.6	T1.5	T1.4	T1.3	T1.2	T1.1	T1.0
34	Reserved	(u16)								
35	T1/T0 msb	(u2,u2,u2)	Reserved				T1.9	T1.8	T0.9	T0.8
36	H0_T0_OUT	(s16)	7	6	5	4	3	2	1	0
37			15	14	13	12	11	10	9	8

y1 :
(T0 Msb << 8 | T0_degC_x8) / 8

y2 :
(T1 Msb << 6 | T1_degC_x8) / 8



(3) Find out y

$$T\_DegC_{(meas)} = (x - x_1) \times \frac{y_2 - y_1}{x_2 - x_1} + y_1 = 29.87\,^{\circ}C. \#$$

Q4. Reconsider Library Efficiency

Is it possible to make the function more efficient with minimal effort (i.e., without venturing into Interrupts, etc.), assuming that the temperature sensor is read repeatedly once a second from the main program? A full function implementation is not expected, explaining the idea will do.

- The calibration register contents are fixed for a particular sensor chip. There's no need to read it over and over again.
- We may set a static flag within HTS221_T_ReadTemp() to indicate if this is the first time being called.
- If yes, read the calibration registers and store the content as static variables.
- If not, query the static variables stored from the first function call to reduce transaction overhead.

Q6. I2C Signal Timing and Condition Mapping

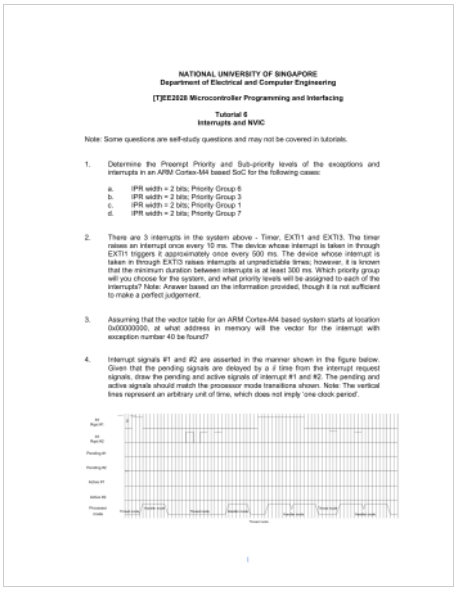
Please go through the answer on your own.

Tutorial 06: Interrupts and NVIC

Saturday, 26 October 2024 10:25 am

[T]EE2028 Tutorial 6 Interrupts

[T]EE2028 Tutorial 6 ...



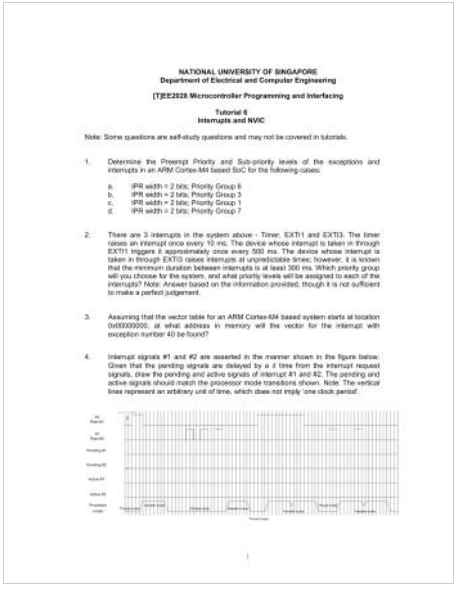
NATIONAL UNIVERSITY OF SINGAPORE
Department of Electrical and Computer Engineering

[T]EE2028 Microcontroller Programming and Interfacing

Tutorial 6
Interrupts and NVIC

Note: Some questions are self-study questions and may not be covered in tutorials.

1. Determine the Preempt Priority and Sub-priority levels of the exceptions and interrupts in an ARM Cortex-M4 based SoC for the following cases:
   a. IPR width = 2 bits; Priority Group 6
   b. IPR width = 2 bits; Priority Group 3
   c. IPR width = 2 bits; Priority Group 1
   d. IPR width = 2 bits; Priority Group 7
2. There are 3 interrupts in the system above - Timer, EXTI1 and EXTI3. The timer raises an interrupt once every 10 ms. The device whose interrupt is taken in through EXTI1 triggers it approximately once every 500 ms. The device whose interrupt is taken in through EXTI3 raises interrupts at unpredictable times; however, it is known that the minimum duration between interrupts is at least 300 ms. Which priority group will you choose for the system, and what priority levels will be assigned to each of the interrupts? Note: Answer based on the information provided; though it is not sufficient to make a perfect judgement.
3. Assuming that the vector table for an ARM Cortex-M4 based system starts at location 0x00000000, at what address in memory will the vector for the interrupt with exception number 40 be found?
4. Interrupt signals #1 and #2 are asserted in the manner shown in the figure below. Given that the pending signals are delayed by a # time from the interrupt request signals, draw the pending and active signals of interrupt #1 and #2. The pending and active signals should match the processor mode transitions shown. Note: The vertical lines represent an arbitrary unit of time, which does not imply 'one clock period'.



NATIONAL UNIVERSITY OF SINGAPORE
Department of Electrical and Computer Engineering

[T]EE2028 Microcontroller Programming and Interfacing

Tutorial 6
Interrupts and NVIC

Note: Some questions are self-study questions and may not be covered in tutorials.

1. Determine the Preempt Priority and Sub-priority levels of the exceptions and interrupts in an ARM Cortex-M4 based SoC for the following cases:
   a. IPR width = 2 bits; Priority Group 6
   b. IPR width = 2 bits; Priority Group 3
   c. IPR width = 2 bits; Priority Group 1
   d. IPR width = 2 bits; Priority Group 7
2. There are 3 interrupts in the system above - Timer, EXTI1 and EXTI3. The timer raises an interrupt once every 10 ms. The device whose interrupt is taken in through EXTI1 triggers it approximately once every 500 ms. The device whose interrupt is taken in through EXTI3 raises interrupts at unpredictable times; however, it is known that the minimum duration between interrupts is at least 300 ms. Which priority group will you choose for the system, and what priority levels will be assigned to each of the interrupts? Note: Answer based on the information provided; though it is not sufficient to make a perfect judgement.
3. Assuming that the vector table for an ARM Cortex-M4 based system starts at location 0x00000000, at what address in memory will the vector for the interrupt with exception number 40 be found?
4. Interrupt signals #1 and #2 are asserted in the manner shown in the figure below. Given that the pending signals are delayed by a # time from the interrupt request signals, draw the pending and active signals of interrupt #1 and #2. The pending and active signals should match the processor mode transitions shown. Note: The vertical lines represent an arbitrary unit of time, which does not imply 'one clock period'.

NATIONAL UNIVERSITY OF SINGAPORE
Department of Electrical and Computer Engineering

[T]EE2028 Microcontroller Programming and Interfacing

Tutorial 6
Interrupts and NVIC

Note: Some questions are self-study questions and may not be covered in tutorials.

1. Determine the Preempt Priority and Sub-priority levels of the exceptions and interrupts in an ARM Cortex-M4 based SoC for the following cases:
   a. IPR width = 2 bits; Priority Group 6
   b. IPR width = 2 bits; Priority Group 3
   c. IPR width = 2 bits; Priority Group 1
   d. IPR width = 2 bits; Priority Group 7
2. There are 3 interrupts in the system above - Timer, EXTI1 and EXTI3. The timer raises an interrupt once every 10 ms. The device whose interrupt is taken in through EXTI1 triggers it approximately once every 500 ms. The device whose interrupt is taken in through EXTI3 raises interrupts at unpredictable times; however, it is known that the minimum duration between interrupts is at least 300 ms. Which priority group will you choose for the system, and what priority levels will be assigned to each of the interrupts? Note: Answer based on the information provided; though it is not sufficient to make a perfect judgement.
3. Assuming that the vector table for an ARM Cortex-M4 based system starts at location 0x00000000, at what address in memory will the vector for the interrupt with exception number 40 be found?
4. Interrupt signals #1 and #2 are asserted in the manner shown in the figure below. Given that the pending signals are delayed by a # time from the interrupt request signals, draw the pending and active signals of interrupt #1 and #2. The pending and active signals should match the processor mode transitions shown. Note: The vertical lines represent an arbitrary unit of time, which does not imply 'one clock period'.

*## The term **"interrupt"** used in my tutorial refers to both **system exceptions** (generated within the processor) and **external interrupts.***

Q1. Priority Group Settings

Table 7.5 Definition of Preempt Priority Field and Subpriority Field in a Priority Level Register in Different Priority Group Settings

Priority Group	Preempt Priority Field	Subpriority Field
0	Bit [7:1]	Bit [0]
1	Bit [7:2]	Bit [1:0]
2	Bit [7:3]	Bit [2:0]
3	Bit [7:4]	Bit [3:0]
4	Bit [7:5]	Bit [4:0]
5	Bit [7:6]	Bit [5:0]
6	Bit [7]	Bit [6:0]
7	None	Bit [7:0]

- IPR width -> Determines the number of bits that can be programmed.
  - In our case: 2 bits
- If N bits are allocated to Preempt and M bits are allocated to Sub-priority:
  - 2^N unique Preempt levels
  - 2^M unique Sub-priority levels

a. IPR width = 2 bits; Priority Group 6: 2 preempt levels, and 2 sub-priority levels.

**Bits that are not implemented, always read as 0*

b. IPR width = 2 bits; Priority Group 3: 4 preempt levels (2^2), and 1 sub-priority level (2^0).

c. IPR width = 2 bits; Priority Group 1: 4 preempt levels, and 1 sub-priority level.
d. IPR width = 2 bits; Priority Group 7: 1 preempt level, and 4 sub-priority levels.

Q2. Interrupt Priority Assignment

- Minimum inter-arrival time is usually used as an indicator of urgency:
  - Timer: 10 ms
  - EXTI1: 500 ms
  - EXTI3: Unpredictable, but at least 300 ms
- Other aspects to consider:
  - Worst-case execution time
    - How long it takes to execute the handler (Callback function)
    - We usually do not wish to have an interrupt with a long handler to interrupt one with shorter handler
  - Long inter-arrival time but critical
    - Example: Emergency button on MRT
    - Some of such emergencies may not be activated in their entire lifetime, but we need to respond to them immediately despite the infinitely long inter-arrival time.

For this question, we assume the timer should have the highest priority, EXTI1 and EXTI3 are comparable. Hence:
Preempt (Timer) < Preempt (EXTI1) = Preempt (EXTI3)

Now, it's time to decide the sub-priorities between EXTI1 and EXTI3, we need more assumptions regarding EXTI3 to proceed:
- If it comes with a **short handler** & **requires a timely response**:
  Sub (EXTI1) > Sub (EXTI3)
- If it comes with a **long handler** & **requires a timely response**:
  - Then it depends on the expected frequency of arrival of EXTI3:
    - Very often: Mabe not a good idea to have it put EXTI1 on pending
      Sub (EXTI1) < Sub (EXTI3)
    - Very rarely: Probably okay to have it put EXTI1 on pending once in a while
      Sub (EXTI1) > Sub (EXTI3)
- If it **doesn't require a very timely response** at all:
  Sub (EXTI1) < Sub (EXTI3)

Summing up: we have 2 preempt levels and at least 2 sub-priority groups per preempt, we could choose:
IPR Width = 2, Priority Group = 6

Timer: Preempt (Timer) = 0x00, Sub (Timer) = 0x00 or 0x40
EXTI1: Preempt (EXTI1) = 0x80, Sub (EXTI1) = 0x80
EXTI1: Preempt (EXTI1) = 0x80, Sub (EXTI1) = 0xC0

OR

Timer: Preempt (Timer) = 0x00, Sub (Timer) = 0x00 or 0x40
EXTI1: Preempt (EXTI1) = 0x80, Sub (EXTI1) = 0xC0
EXTI1: Preempt (EXTI1) = 0x80, Sub (EXTI1) = 0x80

Q3. Interrupt and Exception Vector Table/ ISR (Interrupt Service Routine)/ Handler

When an incoming **system exception** or an **external interrupt**, depending on the architecture, there are two ways for a processor to handle it:

1. No hardware-level classification, all interrupts are taken care of with a **single handler**, within the handler, software implementation is used to identify which ISR should be called.
2. Hardware-level rough classification is implemented, and some interrupts (usually exceptions) are routed to NVIC directly and individually. NVIC then calls the **corresponding handler** for each interrupt.

For ARM Cortex-M4, it mixed uses the two approaches: (Core > Src > stm32l4xx_it.c)
1. Two **shared EXTI handlers** are implemented in the library for **external interrupts** EXTI line [9:5] and EXTI line [15:10]
2. **Dedicated handlers** are implemented in the library for **system exceptions**. (e.g., BusFault, SysTick, NMI, etc.)
3. EXTI0 - EXTI4 are each handled by a dedicated handler, but the Handler function is not implemented in the library.

How does the NVIC know which handler to call? - It has to consult the **vector table**. (STM32L4+ Technical reference manual.pdf)

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	fixed	HardFault	All classes of fault	0x0000 000C
-	0	settable	MemManage	Memory management	0x0000 0010
-	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	6	settable	SysTick	System tick timer	0x0000 003C
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
39	46	settable	USART3	USART3 global interrupt	0x0000 00DC
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000 00E0
41	48	settable	RTC_ALARM	RTC alarms through EXTI line 18 interrupts	0x0000 00E4
42	49	settable	DFSDM1_FLT3	DFSDM1_FLT3 global interrupt	0x0000 00E8

Each entry here indicates a dedicated hardware connection from the source of the interrupt signals to NVIC.
NVIC identifies the corresponding handler to be called through this hash.

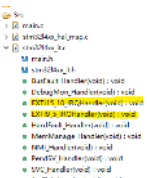
Example:
1. An interrupt signal is coming from the hardware connection **position 40**
2. NVIC consults the table and follows the **address 0x0000 00E0**
3. The **address 0x0000 00E0** contains the starting address of the EXTI15_10 handler function (ISR).

But the above datasheet is not carefully designed (in my opinion), "Position" is not the same as "exception number", as the system exceptions are listed without a specific position. Secondly, the "Priority" here is misleading, please ignore them and except for the "fixed" type of priorities, the rest can always be set by the users.

Hence, to answer this question, we are specifically requesting the vector for interrupt exception 40, i.e., the 40th *interrupt* in the table above. (Not including the reserved address at 0x00!)**

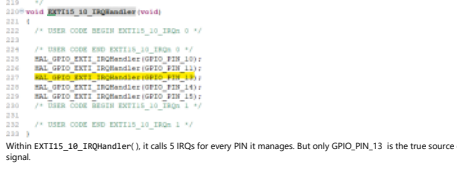
***The 1st address (0x00) is reserved for _estack, which store MSP starting value. The first instruction executed by the proce ssor is LDR SP, _estack, when consulting the table. (This is just for your information)*
For a complete ordered list of exceptions (including the exception stack pointer address, go to Core > Startup > startup_stm32l4s5vitx.s, line 133+).

On a side note, if you find the term "handler function" completely strange to you, read the note here ->



Understanding Handler Functions

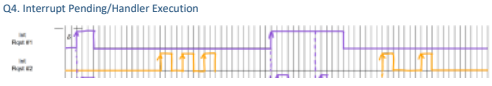
As long as you have loaded the sample project demo_exti_print, you have used EXTI15_10_IRQHandler (../../Core/Src/stm32l4xx_it.c) PB interrupt is routed to EXTI line 13, thus when the interrupt comes in, NVIC will call EXTI15_10_IRQHandler( )



Within `EXTI15_10_IRQHandler()`, it calls 5 IRQs for every PIN it manages. But only GPIO_PIN_13 is the true source of the interrupt signal.

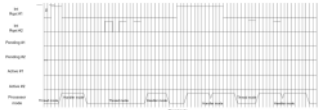
In the `HAL_GPIO_EXTI_IRQHandler()`, line 517 is checking if the input pin is the source of the interrupt. When the input is GPIO_PIN_13 it verifies and confirms the source, then `HAL_GPIO_EXTI_Callback()` will be called and with line 519, the interrupt

Q4. Interrupt Pending/Handler Execution



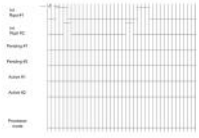
3. Assuming that the vector table for an ARM Cortex-M4 based system starts at location 0x00000000, at what address in memory will the vector for the interrupt with exception number 40 be found?

4. Interrupt signals #1 and #2 are asserted in the manner shown in the figure below. Given that the pending signals are delayed by a δ time from the interrupt request signals, draw the pending and active signals of interrupt #1 and #2. The pending and active signals should match the processor mode transitions shown. Note: The vertical lines represent an arbitrary unit of time, which does not imply 'one clock period'.



5. Interrupt signals #1 and #2 are asserted in the manner shown in the figure below. The following information is given.
   - The pending signals are delayed by 1 x δ from the interrupt request signals.
   - The highest priority pending interrupt is processed right after its pending signal is active by a 2 x δ time delay, of which 1 x δ is for saving the status of the pre-empted main program / handler.
   - Restoring the status of the pre-empted main program / handler upon an interrupt return incurs 1 x δ.
   - There is 1 x δ delay between return from one interrupt handler and the beginning of execution of another.
   - Interrupt #1 has higher preempt priority than interrupt #2.
   - The handler for interrupt #1 takes 6 x δ time units and the handler for interrupt #2 takes 5 x δ time units.

   Draw the pending and active signals of interrupt #1 and #2 and show the processor mode transitions throughout the timeline below.



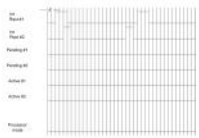
6. Will the ISR for a particular interrupt always take the same duration to execute? Explain.

7. Which register(s) do the following functions modify?

```
(i)  void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority);
(ii) void NVIC_SetPriorityGrouping(uint32_t PriorityGroup);
(iii) uint32_t NVIC_EncodePriority (uint32_t PriorityGroup,
      uint32_t PreemptPriority, uint32_t SubPriority);
```


5. Interrupt signals #1 and #2 are asserted in the manner shown in the figure below. The following information is given.
   - The pending signals are delayed by 1 x δ from the interrupt request signals.
   - The highest priority pending interrupt is processed right after its pending signal is active by a 2 x δ time delay, of which 1 x δ is for saving the status of the pre-empted main program / handler.
   - Restoring the status of the pre-empted main program / handler upon an interrupt return incurs 1 x δ.
   - There is 1 x δ delay between return from one interrupt handler and the beginning of execution of another.
   - Interrupt #1 has higher preempt priority than interrupt #2.
   - The handler for interrupt #1 takes 6 x δ time units and the handler for interrupt #2 takes 5 x δ time units.

   Draw the pending and active signals of interrupt #1 and #2 and show the processor mode transitions throughout the timeline below.



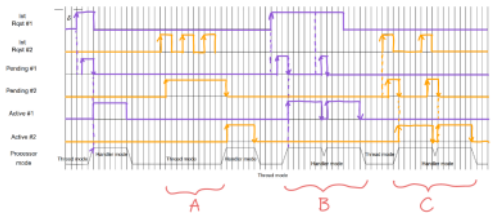
6. Will the ISR for a particular interrupt always take the same duration to execute? Explain.

7. Which register(s) do the following functions modify?

```
(i)  void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority);
(ii) void NVIC_SetPriorityGrouping(uint32_t PriorityGroup);
(iii) uint32_t NVIC_EncodePriority (uint32_t PriorityGroup,
      uint32_t PreemptPriority, uint32_t SubPriority);
```

© [T]EE2026 Teaching Team, ECE, NUS

Q4. Interrupt Pending/Handler Execution



Key takeaways from:
Period A: **Observe Interrupt 2**

- Subsequent incoming interrupt requests will not "promote" the existing pending to a "super pending".
- If the pending state of an interrupt is not cleared, any following interrupt request from the same peripheral will be ignored.
  - 3 requests, 1 pending state, 1 handler execution
- Why are the interrupt requests being ignored? The processor is currently in thread mode, what stops the handler from being called?
  - The only explanation is that Interrupt 2 is not activated at the NVIC's end.
- The activation is only done somewhere between the 3rd request and handler execution. Any problem here?
  - The system is responding to an interrupt from the past, which makes it not relevant anymore in most cases.
  - A crucial step is missing from the activation: clear any possible pending state.

Period B: **Observe Interrupt 1**

- Execute (a properly implemented) handler only clears pending state, and does not clear the interrupts request state.
  - Hence, since the request stays high after the 1st handler execution, which cleared the 1st pending state, it sets the pending state immediately after the delay, and hence handler is executed again.
  - In STM32, the interrupt detection is triggered by raising/falling edges, hence if the pending state is set by raising edge, then we will never see the subsequent pending and handler execution at all.
  - But this question assumed the triggering to be HIGH on the request line (not stated but otherwise unable to explain the 2nd pending)
- A strange observation: The source of Interrupt 1 had no problem clearing the request for the first time, what stops it from doing it now?
  - Could be multiple consecutive requests are raised, the next one coming in before the previous request is cleared.

Period C: **Observe Interrupt 2**

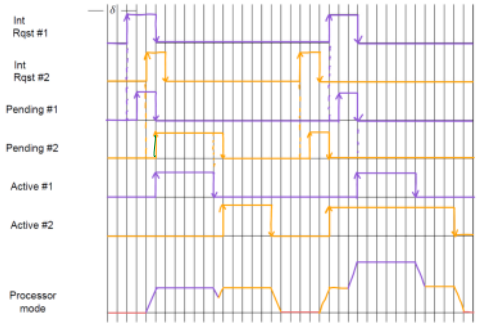
- It doesn't matter if the interrupt handler is currently running, as long as the pending state is not set, and a new request coming, it sets the pending state again, which leads to another handler execution.
- Compare with Period A, and see how the pending state affects the responsiveness to consecutive requests from the same interrupt source.

We will continue with Q5-7 as well as the UART tutorial next week.

Q5.Interrupt Pending/Handler Execution/Priority

Since: Preempt(#1) < Preempt(#2)

- When interrupt #2 request comes in while interrupt #1 is pending, #2 has to wait
- When interrupt #1 request comes in and pended while interrupt #2 handler is executing, interrupt #1 is attended first, during the wait, pay attention to the interrupt #2 active state: it is only reset when the interrupt #2 handler execution is one.



Q6. Worst-Case Execution Time

- We assume 'duration' refers to the actual time spent by the processor in executing the ISR and not including the time the ISR is in a preempted state because of a higher priority handler (though the duration for which the interrupt is considered 'active' includes this time).
- Simple answer to that is: **No**.
- Depending on the exact condition within the device/peripheral that caused the interrupt to be raised, the time taken to deal with it could vary.
  - There could even be multiple conditions that needs to be dealt with simultaneously.
  - There could also be data-dependent if-else scenarios.
  - There is also a possibility that there are loops with variable number of iterations (for example, the number of new data points to be read from a device buffer).
- All these could add to the uncertainty in the execution duration. However, in almost all practical designs, we calculate a certain worst-case execution time (**WCET**), which is an upper bound on the execution duration. We should endeavour to keep the WCET as low as possible by avoiding any heavy-duty processing of data within the handler. Any kind of wait (so-called blocking code), such as delays() or waiting for a certain condition/even should also be avoided.

Q7. Interrupt Configuration through Registers

Interrupt Configuration Registers:

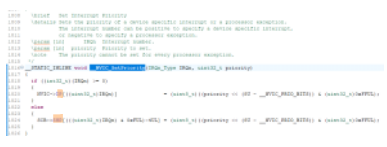
- **ISER** (Set Enable) & **ICER** (Clear Enable)
  - To enable or disable an interrupt. "Disabled" as in NVIC is not set to watch out for a specific interrupt, it does not mean the peripheral is disabled from generating an interrupt signal. (See Q4, Interrupt #2, during Period A)
- **ISPR** (Set Pending) & **ICPR** (Clear Pending)
  - To set pending or clear pending state.
  - If a specific interrupt is already pending, write a "1" to the corresponding ISPR bit has no effect (See Q4, Interrupt # 2, during Period A)
  - If a specific interrupt handler is currently executing, write a "1" to the corresponding ICPR does not stop it.
- **IPR** (Priority)
  - Each 8 bits are used to set the preempt and sub-priority of an interrupt.
- **NVIC_IABR** (Active Bit)
  - Read the corresponding bit to determine if the specific interrupt is currently active
  - You should understand "currently active" as: handler has started executing, but not done yet.
  - Active does not necessarily mean the interrupt handler is executing. It might be preempted by another interrupt with higher preempt. (See Q5, Interrupt #2 active state is set while the handler is interrupted by interrupt #1)
- System Exceptions Registers
  - **SHPRs** (System Handler Priority)
  - **ICSR** (Interrupt control and state), used for non-error type system exceptions, set/clear pending states, determine the current executing exception number.
  - **HCSR** (System handler control and state), used for error/fault type system exceptions. Enable/disable, determine active state.
- Exception-masking Registers
  - "Mask" as in "disguise" or "ignore"
  - These registers can be used if you wish to ignore selective interrupts based on their priority.

Let's now look at codes given:

(i). `void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)`

The function sets the priority of an interrupt or exception.

Hence, it writes to:
- **IPR** for external interrupts
- or **SHPR** for system exceptions



(ii). `void NVIC_SetPriorityGrouping(uint32_t PriorityGroup);`

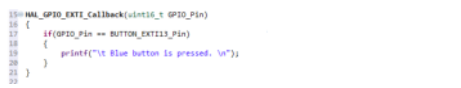
This function set the Priority Grouping in NVIC Interrupt Controller.

Hence, it writes to:
- **PRIGROUP field** in **AIRCR** (Application Interrupt and Reset Control Register)

Within `EXTI15_10_IRQHandler()`, it calls 5 IRQs for every PIN it manages. But only GPIO_PIN_13 is the true source of the interrupt signal.

In the `HAL_GPIO_EXTI_IRQHandler()`, line 517 is checking if the input pin is the source of the interrupt. When the input is `GPIO_PIN_13`, it verifies and confirms the source, then `HAL_GPIO_EXTI_Callback()` will be called and with line 519, the interrupt pending signal is cleared.

Here before the `HAL_GPIO_EXTI_Callback()` it has a "__weak" symbol, this indicates this is a "default" implementation, and serves as a placeholder, if we are to specify what should happen within the ISR, we need to declare a "strong" version in the main.c to "overwrite" the weak version. Just like what we did in the demo code:



(iii). `uint32_t NVIC_EncodePriority (uint32_t PriorityGroup, uint32_t PreemptPriority, uint32_t SubPriority);`

This function ***encode*** the priority for an interrupt. The returned priority value can be used for NVIC_SetPriority(...)

Hence, it does not write to any register, it simply processes some values.

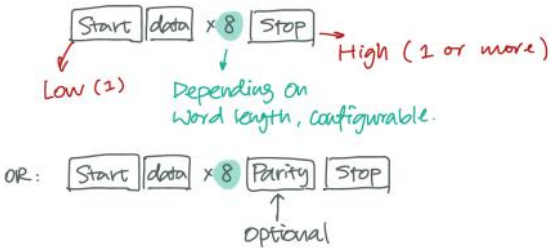
Tutorial 07: Universal Asynchronous Receiver/Transmitter (UART)

2024年11月11日 11:26

[T]EE2028 Tutorial 7 ... → Solution

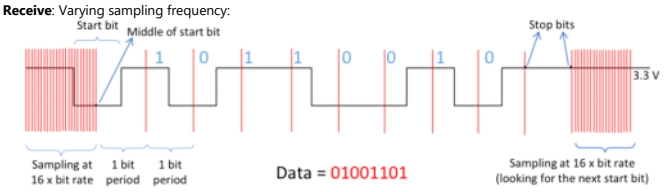
UART Protocol Summary

- **Send**: each character is sent as a frame:



* LSB of the character is sent first.

- **Receive**: Varying sampling frequency:



  - While idling, receiver sampling at **16x bit rate**.
  - Detects signal **LOW** for at least **half-bit** time -> **START**
  - Reads signal around the **middle** of each bit
  - Detects **1-bit time** of **HIGH** -> **STOP**
  - The receiver sampling rate increases again.

NATIONAL UNIVERSITY OF SINGAPORE
Department of Electrical and Computer Engineering

[T]EE2028 Microcontroller Programming and Interfacing

Tutorial 7

Universal Asynchronous Receiver/Transmitter (UART)

Note: Questions 4 and 5 are self-study questions; questions 6 and 7 are non-examinable questions. They will not be covered in the class. Written solutions will be provided.

1. What do you expect to see on an oscilloscope at the transmit pin (TX) of the UART when a byte with value 0xF0 is sent? Assume 1 START bit, 1 STOP bit, no parity, 8 DATA bits.
2. What is the significance of START bit and STOP bit in UART being of opposite logic levels?
3. What is the maximum number of characters that can be sent over a UART with the following settings in one second?
   Baud rate: 10,000. Word length = 8. Parity = None. Stop bits = 1.
4. [Self-study] Two STM32 chips communicate via UART. You need to send one 32-bit integer from STM32_A, which should be printed to the debug console by STM32_B. How can this be accomplished?
5. [Self-study] In the above scenario, if STM32_A wants STM32_B not to send anything while it is performing a critical computation for an arbitrary amount of time. Suggest a mechanism to accomplish this.
6. [Not examinable] How can we check the reliability of the data received through the UART of STM32?
7. [Not examinable] What is the difference between TXE and TC status bits?

NATIONAL UNIVERSITY OF SINGAPORE
Department of Electrical and Computer Engineering

[T]EE2028 Microcontroller Programming and Interfacing

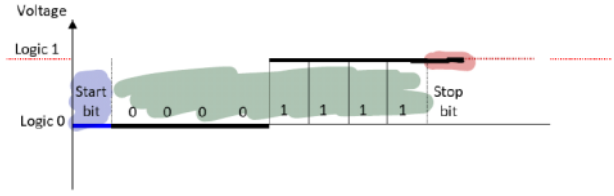
Tutorial 7

Q1. UART Transmission

What do you expect to see on an oscilloscope at the transmit pin (TX) of the UART when a byte with value 0xF0 is sent? Assume 1 START bit, 1 STOP bit, no parity, 8 DATA bits.

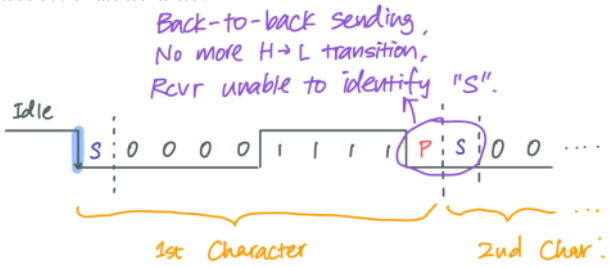
- 0xF0 -> 0b 1111 0000
- UART sends LSB first
- Sending order: 0000 1111

At output TX of UART:



Q2. Asynchronous vs Synchronous Protocols

- **UART** is an **asynchronous** communication mechanism. While the transmitter and receiver have identical settings concerning how long a bit period is, the receiver does not know **when exactly this bit period starts** since there is no shared clock.
- Hence, the **high-to-low transition** of the RX signal at the beginning of the **START bit** is the only way for the receiver to know "that was the beginning of the bit period".
- To guarantee that the START bit causes a transition, the STOP bit should be of the opposite logic level, and the default level of the RX (i.e., when idle) should also be the same logic level as STOP.
  - **If the STOP bit is the same logic level as START (LOW)**, the transition is not guaranteed when two characters are transmitted back-to-back.



  - If the default level is not the same logic level as STOP (STOP HIGH, idle LOW), a time gap between the two characters will cause the beginning of the START bit to be indistinguishable.

NATIONAL UNIVERSITY OF SINGAPORE
Department of Electrical and Computer Engineering

[T]EE2028 Microcontroller Programming and Interfacing

Tutorial 7

Universal Asynchronous Receiver/Transmitter (UART)

Note: Questions 4 and 5 are self-study questions; questions 6 and 7 are non-examinable questions. They will not be covered in the class. Written solutions will be provided.

1. What do you expect to see on an oscilloscope at the transmit pin (TX) of the UART when a byte with value 0xF0 is sent? Assume 1 START bit, 1 STOP bit, no parity, 8 DATA bits.

2. What is the significance of START bit and STOP bit in UART being of opposite logic levels?

3. What is the maximum number of characters that can be sent over a UART with the following settings in one second?
Baud rate: 10,000. Word length = 8. Parity = None. Stop bits = 1.

4. [Self-study] Two STM32 chips communicate via UART. You need to send one 32-bit integer from STM32_A, which should be printed to the debug console by STM32_B. How can this be accomplished?

5. [Self-study] In the above scenario, if STM32_A wants STM32_B not to send anything while it is performing a critical computation for an arbitrary amount of time. Suggest a mechanism to accomplish this.

6. [Not examinable] How can we check the reliability of the data received through the UART of STM32?

7. [Not examinable] What is the difference between TXE and TC status bits?

Q3. Understanding Baud Rate

Given: Baud rate = 10,000. Word length = 8. Parity = None. Stop bits = 1:

- Each Character is sent as a frame of:

- Size of a frame = 1 + 8 + 1 = 10
- The number of frames (characters) that can be sent per second = 10,000 / 10 = 1,000